

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет «Информатика и системы управления»

Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»

Отчет по домашнему заданию

«Oracle»

Выполнил:

студент группы ИУ5-35Б

Бердников Н.О.

Подпись и дата:

Проверил:

Подпись и дата:

Описание проекта Oracle на Go

Проект Oracle представляет собой веб-приложение, разработанное на языке программирования Go, которое предоставляет пользователям возможность получать результаты профессиональных матчей по Dota 2 на основе заданных параметров. Данный проект включает в себя несколько ключевых компонентов и технологий, обеспечивающих его функциональность и производительность.

Архитектура и технологии

Язык программирования: Проект написан на Go, что обеспечивает высокую производительность и простоту в разработке асинхронных приложений.

База данных: Все данные о героях, матчах и профессиональных игроках за последние три года хранятся в PostgreSQL. Это реляционная база данных, которая позволяет эффективно управлять большими объемами информации.

Кэширование: Для оптимизации запросов к базе данных используется Redis. Это решение позволяет значительно ускорить доступ к часто запрашиваемым данным, уменьшая нагрузку на основную базу данных.

Контейнеризация: Проект разворачивается с помощью Docker, что упрощает процесс развертывания и управления зависимостями, а также обеспечивает консистентность окружения.

Функциональные возможности

Регистрация и авторизация: Пользователи могут зарегистрироваться и авторизоваться на сайте. Сессии пользователей хранятся в браузере, что позволяет поддерживать их состояние между запросами.

Асинхронная обработка: Весь проект реализует асинхронную обработку с использованием goroutines, что позволяет эффективно обрабатывать множество запросов одновременно без блокировок.

Тестирование

Для обеспечения качества кода в проекте написаны модульные тесты для всех файлов. Это позволяет выявлять ошибки на ранних стадиях разработки и гарантировать стабильность функционирования приложения.

Модель для предсказания победителя матча в Dota 2

1. Постановка задачи

Разработана модель для предсказания вероятности победы команды в матче Dota 2, основываясь на составах героев, статистике игроков и текущей мете игры.

2. Сбор и подготовка данных

- Источник данных: матчи, полученные через Dota 2 API и OpenDota.
- Предобработка: кодирование героев (one-hot encoding), очистка данных, балансировка классов, добавление мета-информации (винрейты, популярность героев).

3. Разработка модели

- Использовались AutoML-фреймворки для быстрого прототипирования и сравнения моделей.
- Итоговая модель: CatBoost с оптимизацией гиперпараметров.
- Учет синергий и контрпиков героев, а также влияния меты текущего патча.

4. Трекинг и оптимизация

- WandB использовался для отслеживания экспериментов, гиперпараметров и метрик.
- Fine-tuning с использованием байесовской оптимизации и регуляризации.

5. Результаты

- Модель достигла ROC-AUC ~ 0.7 на тестовых данных.
- Важность признаков проанализирована с помощью SHAP.

6. Внедрение

- Реализация сервиса предсказаний на Golang для обеспечения высокой производительности и масштабируемости.
- Интеграция через REST API с веб-интерфейсом для ввода состава команд и получения предсказания.

7. Планы

- Постоянное обновление модели для новых патчей.
- Использование временных характеристик для live-анализа.
- Интеграция с системой аналитики для глубокой игровой статистики.

Исходный код

.gitignore:

.vscode

docker-compose.yml:

version: '3.9'

services:

redis:

image: redis:7.2

ports:

- '6379:6379'

volumes:

- redisdata:/data

db:

build:

context: .

dockerfile: Dockerfile.db

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: 1q2ws3edc4r

POSTGRES_DB: Dota

volumes:

- pgdata:/var/lib/postgresql/data

ports:

- '5432:5432'

app:

build:

context: .

dockerfile: Dockerfile

ports:

- '8080:8080'

depends_on:

- db
- redis

environment:

DB_HOST: db

DB_PORT: 5432

DB_USER: postgres

DB_PASSWORD: 1q2ws3edc4r

DB_NAME: Dota

REDIS_HOST: redis

REDIS_PORT: 6379

volumes:

pgdata:

redisdata:

Dockerfile:

Используем базовый образ с Go

FROM golang:1.23-alpine AS builder

Устанавливаем рабочую директорию

WORKDIR /app

Копируем все файлы проекта в контейнер

COPY . .

Загружаем зависимости и собираем проект

RUN go mod tidy

RUN go build -o main main.go

Используем легковесный образ для финального контейнера

FROM alpine:latest

Устанавливаем зависимости

RUN apk --no-cache add ca-certificates

Создаем рабочую директорию

WORKDIR /app

Копируем собранное приложение из builder-образа

COPY --from=builder /app/main /app/main

Копируем папку с шаблонами

COPY --from=builder /app/web /app/web

Открываем порт (замените 8080 на нужный вам порт)

EXPOSE 8080

Команда для запуска приложения

CMD ["/app/main"]

Dockerfile.db:

Используем официальный образ PostgreSQL

FROM postgres:15

Устанавливаем переменные окружения для пользователя и пароля

```
ENV POSTGRES_USER=postgres
```

```
ENV POSTGRES_PASSWORD=1q2ws3edc4r
```

```
# Копируем скрипты для инициализации
```

```
COPY init.sql /docker-entrypoint-initdb.d/
```

```
COPY load_dota.sql /docker-entrypoint-initdb.d/
```

```
COPY db-dumps/*.dump /docker-entrypoint-initdb.d/
```

```
go.mod:
```

```
module OracleGo
```

```
go 1.23
```

```
toolchain go1.23.2
```

```
require (
```

```
    github.com/gorilla/sessions v1.4.0
```

```
    github.com/lib/pq v1.10.9
```

```
    golang.org/x/crypto v0.28.0
```

```
)
```

```
require (
```


github.com/cespare/xxhash/v2 v2.2.0 // indirect

github.com/dgryski/go-rendezvous v0.0.0-20200823014737-9f7001d12a5f
// indirect

github.com/gorilla/securecookie v1.1.2 // indirect

github.com/redis/go-redis/v9 v9.7.0 // indirect

)

go.sum:

github.com/cespare/xxhash/v2 v2.2.0
h1:DC2CZ1Ep5Y4k3ZQ899DldepgrayRUGE6BBZ/cd9Cj44=

github.com/cespare/xxhash/v2 v2.2.0/go.mod
h1:VGX0DQ3Q6kWi7AoAeZDth3/j3BFtOZR5XLFGgcrjCOs=

github.com/dgryski/go-rendezvous v0.0.0-20200823014737-9f7001d12a5f
h1:IO4WD4F/rVNCu3HqELle0jiPLBs70cWOduZpkS1E78=

github.com/dgryski/go-rendezvous v0.0.0-20200823014737-
9f7001d12a5f/go.mod h1:cuUVRXasLTGF7a8hSLbxyZXjz+1KgoB3wDUb6vlszlc=

github.com/google/gofuzz v1.2.0
h1:xRy4A+RhZaiKj1bPfwQ8sedCA+YS2YcCHW6ec7JMi0=

github.com/google/gofuzz v1.2.0/go.mod
h1:dBl0BpW6vV/+mYPU4Po3pmUjxk6FQPldtuldl/M65Eg=

github.com/gorilla/securecookie v1.1.2
h1:YCIWL56dvtr73r6715mJs5ZvhtnY73hBvEF8kXD8ePA=

github.com/gorilla/securecookie v1.1.2/go.mod
h1:NfCASbcHqRSY+3a8tIWJwsQap2VX5pwzwo4h3eOamfo=

github.com/gorilla/sessions v1.4.0
h1:kplYOp/oi6MG/p5PgXApU8srsSw9tuFbt46Lt7auzqQ=

github.com/gorilla/sessions v1.4.0/go.mod

h1:FLWm50oby91+hl7p/wRxDth9bWSuk0qVL2emc7lT5ik=

github.com/lib/pq v1.10.9

h1:YXG7RB+JlJhP29X+OtkiDnYaXQwpS4JEWq7dtCCRUEw=

github.com/lib/pq v1.10.9/go.mod

h1:AlVN5x4E4T544tWzH6hKfbfQvm3HdbOxrmggDNAPY9o=

github.com/redis/go-redis/v9 v9.7.0

h1:HhLSs+B6O021gwzl+locl0zEDnyNkxMtf/Z3NNBMA9E=

github.com/redis/go-redis/v9 v9.7.0/go.mod

h1:f6zhXITC7JUJIPEiBOTXxJgPLdZcA93Gewl7inzyWw=

golang.org/x/crypto v0.28.0

h1:GBDwsMXVQi34v5CCYUm2jkJvu4cbtru2U4TN2PSyQnw=

golang.org/x/crypto v0.28.0/go.mod

h1:rmgy+3RHxRZMyY0jjAJShp2zgEdOqj2AO7U0pYmeQ7U=

init.sql:

-- Создаем базу данных Dota

CREATE DATABASE Dota;

-- Создаем базу данных DotaTest

CREATE DATABASE DotaTest;

load_dota.sql:

-- Подключаемся к базе данных Dota

\connect Dota;

-- Используем pg_restore для восстановления дампа

-- Предполагается, что Dota.dump находится в формате пользовательского формата

```
\! pg_restore -U postgres -d Dota /docker-entrpoint-initdb.d/Dota.dump;
```

load_dota_test.sql:

-- Подключаемся к базе данных Dota

```
\connect DotaTest;
```

-- Используем pg_restore для восстановления дампа

-- Предполагается, что Dota.dump находится в формате пользовательского формата

```
\! pg_restore -U postgres -d DotaTest /docker-entrpoint-initdb.d/DotaTest.dump;
```

main.go:

```
package main
```

```
import (
```

```
    "OracleGo/internal/auth"
```

```
    "OracleGo/internal/home"
```

```
    "OracleGo/internal/ml"
```

```
    "OracleGo/internal/net"
```

```
"OracleGo/internal/profile"

"OracleGo/internal/statistics"

_ "fmt"

"log"

"net/http"

)

func main() {

    http.HandleFunc("/", home.HomeHandler)


    http.Handle("/login",
net.RedirectIfAuthenticated(http.HandlerFunc(auth.LoginHandler)))

    http.Handle("/register",
net.RedirectIfAuthenticated(http.HandlerFunc(auth.RegisterHandler)))

    http.Handle("/profile",
net.SessionMiddleware(http.HandlerFunc(profile.ProfileHandler)))


    http.HandleFunc("/prediction", ml.PredictionHandler)

    http.HandleFunc("/recommendations", ml.RecommendationsHandler)

    http.HandleFunc("/statistics", statistics.StatisticsHandler)

    http.HandleFunc("/team-analysis", statistics.TeamsHandler)


    log.Println("Server started on :8080")

    log.Fatal(http.ListenAndServe(":8080", nil))

}
```

run_test.bat:

```
set GO_ENV=test
```

```
go test ./internal/...
```

```
set GO_ENV=run
```

run_test.sh:

```
#!/bin/bash
```

```
export GO_ENV=test
```

```
go test ./internal/...
```

```
export GO_ENV=run
```

internal\auth\auth.go:

```
package auth
```

```
import (
```

```

        "golang.org/x/crypto/bcrypt"
    )

    func hashPassword(password string) (string, error) {
        hash, err := bcrypt.GenerateFromPassword([]byte(password),
            bcrypt.DefaultCost)
        if err != nil {
            return "", err
        }
        return string(hash), nil
    }

    func checkPassword(hashPassword string, password string) bool {
        err := bcrypt.CompareHashAndPassword([]byte(hashPassword),
            []byte(password))
        return err == nil
    }

    func findUser(userManager UserManagerInterface, email string) (User, error) {
        return userManager.GetUser(Email{email})
    }

    func ValidateUser(userManager UserManagerInterface, email string, password
    string) bool {
        gottenUser, parsErr := findUser(userManager, email)

```

```
if parsErr != nil {  
  
    if parsErr.Error() == "Пользователь не найден\n" {  
        return false  
    }  
  
    return false  
}
```

```
    if (gettenUser.Email != Email{Value: email}) ||  
!checkPassword(gettenUser.Password.GetValue(), password) {  
  
        return false  
    }  
  
    return true  
}
```

internal\auth\auth_test.go:

```
package auth
```

```
import (
```

```
    "testing"
```

```
)
```

```

func TestHashPassword(t *testing.T) {

    password := "admin"

    hashedPassword, err := hashPassword(password)

    if err != nil {

        t.Errorf("hashPassword returned an error: %v", err)

    }

    if !checkPassword(hashedPassword, password) {

        t.Errorf("checkPassword returned false for correct password")

    }

}

```

```

func TestCheckPassword(t *testing.T) {

    password := "admin"

    hashedPassword, err := hashPassword(password)

    if err != nil {

        t.Errorf("hashPassword returned an error: %v", err)

    }

    tests := []struct {

        name    string

        password string

        expected bool

    }

```



```

    }{
        {"correct password", "admin", true},
        {"incorrect password", "wrongpassword", false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := checkPassword(hashedReader, tt.password)

            if result != tt.expected {
                t.Errorf("checkPassword(%v) = %v; want %v",
tt.password, result, tt.expected)
            }
        })
    }
}

```

```

func TestValidateUser(t *testing.T) {
    tests := []struct {
        name    string
        email    string
        password string
        expected bool
    }{
        {"valid user", "admin@admin.com", "admin", true},
    }
}

```

```

        {"invalid password", "admin@admin.com", "wrongpassword", false},
        {"nonexistent user", "unknown@admin.com", "admin", false},
    }

    manager, _ := NewMockUserDatabaseManager()

    createUser(manager, "admin@admin.com", "admin")

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := ValidateUser(manager, tt.email, tt.password)

            if result != tt.expected {
                t.Errorf("ValidateUser(%v, %v) = %v; want %v", tt.email,
                    tt.password, result, tt.expected)
            }
        })
    }
}

```

internal\auth\dbUsers.go:

```
package auth
```

```
import (
```

```
    "OracleGo/internal/db"
```

```
    "errors"
```

```
    "fmt"
```

```
_ "fmt"
```

```
_ "github.com/lib/pq"
```

```
)
```

```
type Email struct {
```

```
    Value string
```

```
}
```

```
func (e Email) GetValue() string {
```

```
    return e.Value
```

```
}
```

```
type Password struct {
```

```
    Value string
```

```
}
```

```
func (p Password) GetValue() string {
```

```
    return p.Value
```

```
}
```

```
type User struct {
```

```
    Email    Email
```

```

        Password Password
    }

type UserManagerInterface interface {
    GetUser(email Email) (User, error)
    AddUsers(emails []Email, passwords []Password) error
}

type UsersDatabaseManager struct {
    dbManager db.DatabaseManager
}

func NewUsersDatabaseManager() (*UsersDatabaseManager, error) {
    dbManager, err := db.NewDatabaseManager()
    if err != nil {
        return nil, err
    }
    return &UsersDatabaseManager{dbManager: *dbManager}, nil
}

func (usersDbManager *UsersDatabaseManager) getUserAsync(email Email,
resultChan chan<- User, errorChan chan<- error) {
    userRows, err := usersDbManager.dbManager.GetRows("users",
[]string{"email", "password"}, map[string][]string{"email": {email.Value}})

```

```

if err != nil {

    errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

    return

}

if len(userRows) == 0 {

    errorChan <- errors.New("Пользователь не найден\n")

    return

}

result := User{

    Email:  Email{userRows[0][0]},

    Password: Password{userRows[0][1]},

}

// Отправка результата в канал

resultChan <- result

}

func (usersDbManager *UsersDatabaseManager) GetUser(email Email) (User,
error) {

    resultChan := make(chan User)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов

    go usersDbManager.getUserAsync(email, resultChan, errorChan)

```

```

// Ожидание результата или ошибки

select {

case result := <-resultChan:

    return result, nil

case err := <-errorChan:

    return User{}, err

}

}

func (usersDbManager *UsersDatabaseManager) addUsersAsync(emails []Email,
passwords []Password, errorChan chan<- error) {

    emailsValues := db.ValuesFromAny(emails)

    passwordsValues := db.ValuesFromAny(passwords)

    err := usersDbManager.dbManager.AddRows("users",
map[string][]string{"email": emailsValues, "password": passwordsValues})

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

    // Отправка nil в errorChan для обозначения успешного завершения

    errorChan <- nil

```

```
}
```

```
func (usersDbManager *UsersDatabaseManager) AddUsers(emails []Email,  
passwords []Password) error {
```

```
    errorChan := make(chan error)
```

```
    // Запуск асинхронной функции с передачей канала ошибок
```

```
    go usersDbManager.addUsersAsync(emails, passwords, errorChan)
```

```
    // Ожидание результата или ошибки
```

```
    if err := <-errorChan; err != nil {
```

```
        return err
```

```
    }
```

```
    return nil
```

```
}
```

```
internal\auth\dbUsers_test.go:
```

```
package auth
```

```
import (
```

```
    "OracleGo/internal/db"
```

```
    "fmt"
```

```
    "testing"
```

)

```
func setupTestDatabaseManager() (*UsersDatabaseManager, error) {  
    dbManager, err := db.NewDatabaseManager()  
    if err != nil {  
        return nil, fmt.Errorf("failed to setup test database manager: %v",  
err)  
    }  
  
    _ = dbManager.DeleteRows("users", map[string][]string{})  
  
    return &UsersDatabaseManager{dbManager: *dbManager}, nil  
}
```

```
func TestAddUsers(t *testing.T) {  
    manager, err := setupTestDatabaseManager()  
    if err != nil {  
        t.Fatalf("Failed to set up test database manager: %v", err)  
    }  
  
    email := Email{Value: "testuser@example.com"}  
    password := Password{Value: "hashedpassword123"}  
  
    err = manager.AddUsers([]Email{email}, []Password{password})
```



```

    if err != nil {

        t.Errorf("AddUsers returned an error: %v", err)

    }

    user, err := manager.GetUser(email)

    if err != nil {

        t.Errorf("GetUser returned an error for existing user: %v", err)

    }

    if user.Email.GetValue() != email.GetValue() || user.Password.GetValue() !=
password.GetValue() {

        t.Errorf("User data does not match added values, got: %+v, want:
%+v", user, User{Email: email, Password: password})

    }

}

func TestGetUser_NotFound(t *testing.T) {

    manager, err := setupTestDatabaseManager()

    if err != nil {

        t.Fatalf("Failed to set up test database manager: %v", err)

    }

    _, err = manager.GetUser(Email{Value: "nonexistent@example.com"})

    if err == nil || err.Error() != "Пользователь не найден\n" {

```

```

        t.Errorf("Expected 'Пользователь не найден' error, got: %v", err)
    }
}

```

```

func TestAddUsers_MultipleUsers(t *testing.T) {
    manager, err := setupTestDatabaseManager()

    if err != nil {
        t.Fatalf("Failed to set up test database manager: %v", err)
    }

    emails := []Email{{Value: "user1@example.com"}, {Value:
"user2@example.com"}}

    passwords := []Password{{Value: "hashedpassword1"}, {Value:
"hashedpassword2"}}

    err = manager.AddUsers(emails, passwords)

    if err != nil {
        t.Errorf("AddUsers returned an error: %v", err)
    }

    for i, email := range emails {
        user, err := manager.GetUser(email)

        if err != nil {
            t.Errorf("GetUser returned an error for added user %v: %v",
email.GetValue(), err)
        }

        if user.Password.GetValue() != passwords[i].GetValue() {

```

```

                                t.Errorf("User password does not match added value for %v,
got: %s, want: %s", email.GetValue(), user.Password.GetValue(),
passwords[i].GetValue())
                                }
                                }
                                }

```

internal\auth\loginPageHandler.go:

```
package auth
```

```
import (
    "OracleGo/internal/net"
    _ "fmt"
    "log"
    "net/http"
)

```

```
func LoginHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]interface{}{}

    if r.Method == http.MethodPost {
        email := r.FormValue("email")
        inputPassword := r.FormValue("password")
    }
}

```

```

userManager, err := NewUsersDatabaseManager()

if err != nil {

    log.Fatalf("Не удалось создать DataBaseManager: %v", err)

}

isValidUser := ValidateUser(userManager, email, inputPassword)

if isValidUser {

    err := net.SaveSession(w, r, email)

    if err == nil {

        http.Redirect(w, r, "/profile", http.StatusSeeOther)

        return

    } else {

        http.Error(w, "Error saving session",
http.StatusInternalServerError)

        return

    }

} else {

    data["ErrorMessage"] = "Incorrect email or password"

}

}

net.RenderTemplate(w, r, "login.html", data)

}

```

```
internal\auth\loginPageHandler_test.go:
```

```
package auth
```

```
import (
```

```
    "net/http"
```

```
    "net/http/httptest"
```

```
    "strings"
```

```
    "testing"
```

```
)
```

```
func setupTestUser(t *testing.T) {
```

```
    manager, err := NewUsersDatabaseManager()
```

```
    if err != nil {
```

```
        t.Fatalf("Не удалось создать UsersDatabaseManager: %v", err)
```

```
    }
```

```
    AddUser(manager, "test@example.com", "password", "password")
```

```
}
```

```
// Тест успешной авторизации
```

```
func TestLoginHandler_Success(t *testing.T) {
```

```
    setupTestUser(t)
```

```
    // Создание HTTP-запроса
```

```
    reqBody :=
strings.NewReader("email=test@example.com&password=password")

    req := httptest.NewRequest(http.MethodPost, "/login", reqBody)

    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")

    w := httptest.NewRecorder()

    // Вызов обработчика
    LoginHandler(w, req)

    // Проверка результата
    resp := w.Result()

    defer resp.Body.Close()

    // Проверяем, что был выполнен редирект на профиль
    if resp.StatusCode != http.StatusSeeOther {

        t.Errorf("Ожидался статус %v, но получен %v", http.StatusSeeOther,
resp.StatusCode)

    }

    if loc := resp.Header.Get("Location"); loc != "/profile" {

        t.Errorf("Ожидался редирект на /profile, но получен %s", loc)

    }

}
```

// Тест с неверными данными авторизации

```
func TestLoginHandler_InvalidCredentials(t *testing.T) {  
  
    reqBody :=  
strings.NewReader("email=wrong@example.com&password=wrongpassword")  
  
    req := httptest.NewRequest(http.MethodPost, "/login", reqBody)  
  
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")  
  
    w := httptest.NewRecorder()  
  
    LoginHandler(w, req)  
  
    resp := w.Result()  
  
    defer resp.Body.Close()  
  
    // Проверяем, что обработчик не выполнил редирект и вернул  
    страницу с ошибкой  
  
    if resp.StatusCode != http.StatusOK {  
  
        t.Errorf("Ожидался статус %v, но получен %v", http.StatusOK,  
resp.StatusCode)  
  
    }  
  
}
```

// Тест с методом GET, чтобы убедиться, что возвращается страница логина

```
func TestLoginHandler_GetMethod(t *testing.T) {  
  
    req := httptest.NewRequest(http.MethodGet, "/login", nil)  
  
    w := httptest.NewRecorder()
```

```

LoginHandler(w, req)

resp := w.Result()

defer resp.Body.Close()

// Проверка успешного рендеринга страницы логина
if resp.StatusCode != http.StatusOK {
    t.Errorf("Ожидался статус %v, но получен %v", http.StatusOK,
resp.StatusCode)
}
}

```

internal\auth\regist.go:

```
package auth
```

```

func createUser(userManager UserManagerInterface, email string, password
string) error {
    hashedPassword, hashErr := hashPassword(password)

    if hashErr != nil {
        return hashErr
    }

    return userManager.AddUsers([]Email{Email{email}},
[]Password{Password{hashedPassword}})
}

```



```
}
```

```
func AddUser(userManager UserManagerInterface, email string, password string,  
confirmPassword string) bool {
```

```
    if password != confirmPassword {
```

```
        return false
```

```
    }
```

```
    _, parsErr := findUser(userManager, email)
```

```
    if parsErr != nil {
```

```
        if parsErr.Error() == "Пользователь не найден\n" {
```

```
            return createUser(userManager, email, password) == nil
```

```
        }
```

```
        return false
```

```
    }
```

```
    return false
```

```
}
```

internal\auth\registerPageHandler.go:

```
package auth
```

```
import (
```

```
    "OracleGo/internal/net"
```

```
    _ "fmt"
```

```
    "log"
```

```
    "net/http"
```

```
)
```

```
func RegisterHandler(w http.ResponseWriter, r *http.Request) {
```

```
    data := map[string]interface{}{}
```

```
    if r.Method == http.MethodPost {
```

```
        email := r.FormValue("email")
```

```
        inputPassword := r.FormValue("password")
```

```
        confirmPassword := r.FormValue("confirm-password")
```

```
        userManager, err := NewUsersDatabaseManager()
```

```
        if err != nil {
```

```
            log.Fatalf("Не удалось создать DataBaseManager: %v", err)
```

```
        }
```

```
        isCreateUser := AddUser(userManager, email, inputPassword,  
confirmPassword)
```

```
        if isCreateUser {  
            http.Redirect(w, r, "/login", http.StatusSeeOther)  
  
        } else {  
            data["ErrorMessage"] = "Incorrect email or password"  
        }  
    }  
  
    net.RenderTemplate(w, r, "register.html", data)  
}
```

internal\auth\registerPageHandler_test.go:

```
package auth
```

```
import (  
    "io"  
    "net/http"  
    "net/http/httptest"  
    "strings"  
    "testing"  
)
```

```
func TestRegisterHandler_Success(t *testing.T) {  
  
    // Подготовка тестовых данных  
  
    setupTestUser(t)  
  
  
    // Создание HTTP-запроса для регистрации  
  
    reqBody :=  
strings.NewReader("email=newuser@example.com&password=password&confirm-password=password")  
  
    req := httptest.NewRequest(http.MethodPost, "/register", reqBody)  
  
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")  
  
    w := httptest.NewRecorder()  
  
  
    // Вызов обработчика  
  
    RegisterHandler(w, req)  
  
  
    // Получаем результат  
  
    resp := w.Result()  
  
    defer resp.Body.Close()  
  
  
    // Проверяем статус ответа (должен быть редирект на /login)  
  
    if resp.StatusCode != http.StatusSeeOther {  
  
        t.Errorf("Ожидался статус %v, но получен %v", http.StatusSeeOther,  
resp.StatusCode)  
  
    }  
}
```

```

// Проверяем редирект на /login
loc := resp.Header.Get("Location")

if loc != "/login" {
    t.Errorf("Ожидался редирект на /login, но получен %s", loc)
}
}

func TestRegisterHandler_PasswordsDoNotMatch(t *testing.T) {
    // Создание HTTP-запроса для регистрации с разными паролями
    reqBody :=
strings.NewReader("email=newuser@example.com&password=password&confirm-password=wrongpassword")

    req := httptest.NewRequest(http.MethodPost, "/register", reqBody)
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")
    w := httptest.NewRecorder()

    // Вызов обработчика
    RegisterHandler(w, req)

    // Получаем результат
    resp := w.Result()
    defer resp.Body.Close()

    // Проверяем, что не произошло редиректа, а ошибка отображается на
    // странице
    if resp.StatusCode == http.StatusSeeOther {

```

```
        t.Errorf("Ожидался статус не редиректа, но получен %v",  
resp.StatusCode)
```

```
    }
```

```
    // Проверяем, что сообщение об ошибке присутствует
```

```
    bodyBytes, _ := io.ReadAll(resp.Body)
```

```
    if !strings.Contains(string(bodyBytes), "Incorrect email or password") {
```

```
        t.Errorf("Ожидалось сообщение об ошибке, но его нет")
```

```
    }
```

```
}
```

```
func TestRegisterHandler_EmailAlreadyExists(t *testing.T) {
```

```
    // Добавляем пользователя с таким email для симуляции ситуации,  
    когда email уже существует
```

```
    setupTestUser(t)
```

```
    // Создание HTTP-запроса для регистрации с уже существующим email
```

```
    reqBody :=
```

```
strings.NewReader("email=test@example.com&password=password&confirm-  
password=password")
```

```
    req := http.NewRequest(http.MethodPost, "/register", reqBody)
```

```
    req.Header.Set("Content-Type", "application/x-www-form-urlencoded")
```

```
    w := http.NewRecorder()
```

```
    // Вызов обработчика
```

```
    RegisterHandler(w, req)
```

```

// Получаем результат

resp := w.Result()

defer resp.Body.Close()


// Проверяем, что не произошло редиректа

if resp.StatusCode == http.StatusSeeOther {

    t.Errorf("Ожидался статус не редиректа, но получен %v",
resp.StatusCode)

}


// Проверяем, что ошибка связана с уже существующим email

bodyBytes, _ := io.ReadAll(resp.Body)

if !strings.Contains(string(bodyBytes), "Incorrect email or password") {

    t.Errorf("Ожидалось сообщение об ошибке, но его нет")

}

}

func TestRegisterHandler_GetRequest(t *testing.T) {

    // Создание GET-запроса на страницу регистрации

    req := httptest.NewRequest(http.MethodGet, "/register", nil)

    w := httptest.NewRecorder()


    // Вызов обработчика

    RegisterHandler(w, req)

```

```
// Получаем результат

resp := w.Result()

defer resp.Body.Close()


// Проверяем, что запрос вернул статус 200 OK

if resp.StatusCode != http.StatusOK {

    t.Errorf("Ожидался статус %v, но получен %v", http.StatusOK,
resp.StatusCode)

}


// Дополнительно можно проверить, что тело ответа содержит форму
регистрации

bodyBytes, _ := io.ReadAll(resp.Body)

bodyStr := string(bodyBytes)


// Проверяем, что в теле ответа есть поля для ввода email и пароля

if !strings.Contains(bodyStr, "name=\"email\"") {

    t.Errorf("Ожидалось поле для ввода email, но его нет в теле
ответа")

}


if !strings.Contains(bodyStr, "name=\"password\"") {

    t.Errorf("Ожидалось поле для ввода пароля, но его нет в теле
ответа")

}
```



```
}
```

```
    if !strings.Contains(bodyStr, "name=\"confirm-password\"") {  
        t.Errorf("Ожидалось поле для ввода подтверждения пароля, но  
его нет в теле ответа")  
    }  
}
```

internal\auth\regist_test.go:

```
package auth
```

```
import (
```

```
    "errors"
```

```
    "testing"
```

```
)
```

```
type MockUserDatabaseManager struct {
```

```
    users map[string]string
```

```
}
```

```
func NewMockUserDatabaseManager() (*MockUserDatabaseManager, error) {
```

```
    return &MockUserDatabaseManager{users: make(map[string]string)}, nil
```

```
}
```

```

func (m *MockUserDatabaseManager) GetUser(email Email) (User, error) {
    password, exists := m.users[email.Value]
    if !exists {
        return User{}, errors.New("Пользователь не найден\n")
    }
    return User{
        Email: email,
        Password: Password{Value: password},
    }, nil
}

```

```

func (m *MockUserDatabaseManager) AddUsers(emails []Email, passwords
[]Password) error {
    if len(emails) != len(passwords) {
        return errors.New("Несоответствие количества email и паролей")
    }
    for i, email := range emails {
        m.users[email.Value] = passwords[i].GetValue()
    }
    return nil
}

```

```

func TestCreateUser(t *testing.T) {

```

```

manager, _ := NewMockUserDatabaseManager()

email := "admin@admin.com"

password := "admin"


err := createUser(manager, email, password)

if err != nil {

    t.Errorf("createUser returned an error: %v", err)

}


user, err := manager.GetUser(Email{Value: email})

if err != nil {

    t.Errorf("User was not created: %v", err)

}


if !checkPassword(user.Password.GetValue(), password) {

    t.Errorf("Stored password does not match the original")

}

}


func TestAddUser(t *testing.T) {

    tests := []struct {

        name      string

        email     string

```

```

password    string

confirmPassword string

setup       func(manager *MockUserDatabaseManager)

expected    bool
}

{
    name:      "successful user creation",
    email:     "admin@admin.com",
    password:  "admin",
    confirmPassword: "admin",
    setup:     func(manager *MockUserDatabaseManager) {},
    expected:  true,
},

{
    name:      "passwords do not match",
    email:     "admin@admin.com",
    password:  "admin",
    confirmPassword: "notadmin",
    setup:     func(manager *MockUserDatabaseManager) {},
    expected:  false,
},

{
    name:      "user already exists",

```

```

        email:      "admin@admin.com",
        password:    "admin",
        confirmPassword: "admin",
        setup: func(manager *MockUserDatabaseManager) {
            hashedPassword, _ := hashPassword("admin")
            manager.users["admin@admin.com"] = hashedPassword
        },
        expected: false,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        manager, _ := NewMockUserDatabaseManager()
        tt.setup(manager)

        result := AddUser(manager, tt.email, tt.password,
tt.confirmPassword)

        if result != tt.expected {
            t.Errorf("AddUser(%v, %v, %v) = %v; want %v", tt.email,
tt.password, tt.confirmPassword, result, tt.expected)
        }
    })
}

```

```
}
```

```
internal\db\db.go:
```

```
package db
```

```
import (
```

```
    "database/sql"
```

```
    "fmt"
```

```
    _ "github.com/lib/pq"
```

```
)
```

```
type DB struct {
```

```
    conn *sql.DB
```

```
}
```

```
func NewDB(host, port, user, password, dbname string) (*DB, error) {
```

```
    connStr := fmt.Sprintf("host=%s port=%s user=%s password=%s  
dbname=%s sslmode=disable", host, port, user, password, dbname)
```

```
    db, err := sql.Open("postgres", connStr)
```

```
    if err != nil {
```

```
        return nil, err
```

```
    }
```

```

    err = db.Ping()

    if err != nil {

        return nil, err

    }

    return &DB{conn: db}, nil
}

func (db *DB) Close() error {

    return db.conn.Close()

}

func (db *DB) Query(query string, args ...interface{}) (*sql.Rows, error) {

    rows, err := db.conn.Query(query, args...)

    if err != nil {

        return nil, err

    }

    return rows, nil

}

func (db *DB) Exec(query string, args ...interface{}) (sql.Result, error) {

    result, err := db.conn.Exec(query, args...)

```

```
    if err != nil {  
        return nil, err  
    }  
    return result, nil  
}
```

internal\db\dbManager.go:

```
package db
```

```
import (  
    "OracleGo/internal/redis"  
    "fmt"  
    "log"  
    "os"  
    "strings"  
    "sync"  
    "time"  
    _ "time"  
  
    _ "github.com/lib/pq"  
)
```



```
type DatabaseManager struct {
```

```
    dbName string
```

```
    db    DB
```

```
    dbLock sync.Mutex
```

```
    redis redis.RedisManager
```

```
}
```

```
func NewDatabaseManager() (*DatabaseManager, error) {
```

```
    dbName := "Dota"
```

```
    if os.Getenv("GO_ENV") == "test" {
```

```
        dbName = "DotaTest"
```

```
    }
```

```
    db, err := NewDB("db", "5432", "postgres", "1q2ws3edc4r", dbName)
```

```
    if err != nil {
```

```
        return nil, err
```

```
    }
```

```
    return &DatabaseManager{dbName: dbName, db: *db, redis:  
*redis.NewRedisManager()}, nil
```

```
}
```

```
func (dbManager *DatabaseManager) BuildSQLQuery(tableName string, params  
[]string, args map[string][]string) string {
```

```
    query := "SELECT " + strings.Join(params, ", ") + " FROM " + tableName
```

```

    if len(args) > 0 {
        var whereClauses []string
        for key, values := range args {
            if len(values) > 0 {
                placeholders := make([]string, len(values))
                for i := range values {
                    placeholders[i] = fmt.Sprintf("%s", values[i])
                }
                whereClauses = append(whereClauses, fmt.Sprintf("%s
IN (%s)", key, strings.Join(placeholders, ", ")))
            }
        }
        if len(whereClauses) > 0 {
            query += " WHERE " + strings.Join(whereClauses, " AND ")
        }
    }

    return query
}

```

```

func (dbManager *DatabaseManager) GetRows(tableName string, params
[]string, args map[string][]string) ([][]string, error) {

```

```

    // Составляем запрос с плейсхолдерами

```

```
    query := fmt.Sprintf("SELECT %s FROM %s", strings.Join(params, " "),
tableName)
```

```
var whereClauses []string
```

```
var queryArgs []interface{}
```

```
i := 1
```

```
for key, values := range args {
```

```
    placeholders := make([]string, len(values))
```

```
    for j, value := range values {
```

```
        placeholders[j] = fmt.Sprintf("$%d", i)
```

```
        queryArgs = append(queryArgs, value)
```

```
        i++
```

```
    }
```

```
    whereClauses = append(whereClauses, fmt.Sprintf("%s IN (%s)", key,
strings.Join(placeholders, " ")))
```

```
}
```

```
if len(whereClauses) > 0 {
```

```
    query += " WHERE " + strings.Join(whereClauses, " AND ")
```

```
}
```

```
cachedQuery := dbManager.dbName + fmt.Sprintf(query, queryArgs)
```

```
if tableName != "users" {
```

```
    var cachedData [][]string
```

```
    err := dbManager.redis.GetCachedData(cachedQuery, &cachedData)
```

```
        if err == nil {

            fmt.Println("данные есть в кеше", cachedQuery, cachedData)

            //log.Fatal("данные есть в кеше", cachedQuery)

            return cachedData, nil

        }

        fmt.Println("данных нет в кеше", cachedQuery)

        //log.Fatal("данных нет в кеше", cachedQuery)

    }

    // Выполняем подготовленный запрос
    rows, err := dbManager.db.Query(query, queryArgs...)

    if err != nil {

        log.Fatalf("Ошибка выполнения запроса: %v", err)

    }

    defer rows.Close()

    var result [][]string

    columns, err := rows.Columns()

    if err != nil {

        return nil, err

    }

    for rows.Next() {
```

```
values := make([]interface{}, len(columns))  
valuePtrs := make([]interface{}, len(columns))
```

```
for i := range values {  
    valuePtrs[i] = &values[i]  
}
```

```
err := rows.Scan(valuePtrs...)  
if err != nil {  
    return nil, err  
}
```

```
row := make([]string, len(columns))  
for i, val := range values {  
    if val != nil {  
        row[i] = fmt.Sprintf("%v", val)  
    } else {  
        row[i] = "NULL"  
    }  
}
```

```
result = append(result, row)
```

```
}
```

```

        if err = rows.Err(); err != nil {

            return nil, err

        }

        dbManager.dbLock.Lock()

        defer dbManager.dbLock.Unlock()

        if tableName != "users" {

            dbManager.redis.CacheData(cachedQuery, result, 10*time.Minute)

            if err != nil {

                fmt.Printf("Ошибка при кэшировании данных: %v/n", err)

                log.Fatalf("Ошибка при кэшировании данных: %v", err)

            }

        }

        return result, nil

    }

```

// Модифицированный метод для безопасного добавления строк

```

func (dbManager *DatabaseManager) AddRows(tableName string, args
map[string][]string) error {

    dbManager.dbLock.Lock()

    defer dbManager.dbLock.Unlock()

    columns := make([]string, 0, len(args))

```

```

for col := range args {

    columns = append(columns, col)

}

numRows := len(args[columns[0]])

for _, values := range args {

    if len(values) != numRows {

        return fmt.Errorf("mismatched number of values in columns")

    }

}

// Формируем запрос с плейсхолдерами
valuePlaceholders := make([]string, numRows)

var queryArgs []interface{}

argIndex := 1

for i := 0; i < numRows; i++ {

    rowPlaceholders := make([]string, len(columns))

    for j, col := range columns {

        rowPlaceholders[j] = fmt.Sprintf("$%d", argIndex)

        queryArgs = append(queryArgs, args[col][i])

        argIndex++

    }

    valuePlaceholders[i] = fmt.Sprintf("(%s)",
strings.Join(rowPlaceholders, ", "))

```

```
}
```

```
    query := fmt.Sprintf("INSERT INTO \"%s\" (%s) VALUES %s", tableName,  
strings.Join(columns, " "), strings.Join(valuePlaceholders, " "))
```

```
    // Выполняем подготовленный запрос
```

```
    _, err := dbManager.db.Exec(query, queryArgs...)
```

```
    if err != nil {
```

```
        return fmt.Errorf("error inserting rows: %v", err)
```

```
    }
```

```
    return nil
```

```
}
```

```
func (dbManager *DatabaseManager) DeleteRows(tableName string, args  
map[string][]string) error {
```

```
    dbManager.dbLock.Lock()
```

```
    defer dbManager.dbLock.Unlock()
```

```
    // Начинаем с основного запроса
```

```
    query := fmt.Sprintf("DELETE FROM %s", tableName)
```

```
    // Формируем условия WHERE с использованием плейсхолдеров
```

```
    var whereClauses []string
```

```
    var queryArgs []interface{}
```

```
    i := 1
```



```

for key, values := range args {
    placeholders := make([]string, len(values))
    for j, value := range values {
        placeholders[j] = fmt.Sprintf("$%d", i)
        queryArgs = append(queryArgs, value)
        i++
    }

    whereClauses = append(whereClauses, fmt.Sprintf("%s IN (%s)", key,
strings.Join(placeholders, ", ")))
}

// Добавляем условия WHERE, если они есть
if len(whereClauses) > 0 {
    query += " WHERE " + strings.Join(whereClauses, " AND ")
}

// Выполняем запрос
_, err := dbManager.db.Exec(query, queryArgs...)
if err != nil {
    return fmt.Errorf("ошибка удаления строк: %v", err)
}

return nil
}

```

internal\db\dbManager_test.go:

```
package db
```

```
import (
```

```
    "os"
```

```
    "testing"
```

```
)
```

```
// setupTestDBManager инициализирует тестовый `DatabaseManager`.
```

```
func setupTestDBManager(t *testing.T) *DatabaseManager {
```

```
    t.Helper()
```

```
    // Устанавливаем переменную окружения для тестовой базы данных
```

```
    os.Setenv("GO_ENV", "test")
```

```
    dbManager, err := NewDatabaseManager()
```

```
    if err != nil {
```

```
        t.Fatalf("Не удалось создать DatabaseManager: %v", err)
```

```
    }
```

```
    return dbManager
```

```
}
```

// TestBuildSQLQuery проверяет генерацию SQL-запросов с различными параметрами.

```
func TestBuildSQLQuery(t *testing.T) {
```

```
    dbManager := setupTestDBManager(t)
```

```
    // Пример запроса с условиями
```

```
    tableName := "users"
```

```
    params := []string{"email", "password"}
```

```
    args := map[string][]string{"email": {"test@example.com"}}
```

```
    expectedQuery := "SELECT email, password FROM users WHERE email IN ('test@example.com')"
```

```
    query := dbManager.BuildSQLQuery(tableName, params, args)
```

```
    if query != expectedQuery {
```

```
        t.Errorf("Ожидался запрос %v, но получен %v", expectedQuery, query)
```

```
    }
```

```
}
```

// TestAddRows проверяет добавление строк в таблицу `users`.

```
func TestAddRows(t *testing.T) {
```

```
    dbManager := setupTestDBManager(t)
```

```
    defer dbManager.db.Close()
```

```
    // Добавление тестового пользователя
```

```

args := map[string][]string{
    "email": {"test@example.com"},
    "password": {"password"},
}

err := dbManager.AddRows("users", args)

if err != nil {
    t.Fatalf("Ошибка при добавлении строки в таблицу users: %v", err)
}

// Удаление тестового пользователя после добавления
defer dbManager.DeleteRows("users", map[string][]string{"email":
{"test@example.com"}})

// Проверяем, что данные добавлены
rows, err := dbManager.GetRows("users", []string{"email", "password"},
map[string][]string{"email": {"test@example.com"}})

if err != nil {
    t.Fatalf("Ошибка при получении строки из таблицы users: %v", err)
}

if len(rows) != 1 || rows[0][0] != "test@example.com" || rows[0][1] !=
"password" {
    t.Errorf("Ожидались данные email=test@example.com,
password=password, но получены %v", rows)
}

```

```

    }

}

// TestGetRows проверяет выборку данных из таблицы `users`.
func TestGetRows(t *testing.T) {

    dbManager := setupTestDBManager(t)

    defer dbManager.db.Close()

    // Добавляем тестового пользователя

    err := dbManager.AddRows("users", map[string][]string{"email":
{"test@example.com"}, "password": {"password"}})

    if err != nil {

        t.Fatalf("Ошибка при добавлении тестового пользователя: %v",
err)

    }

    defer dbManager.DeleteRows("users", map[string][]string{"email":
{"test@example.com"}})

    // Получаем данные пользователя

    rows, err := dbManager.GetRows("users", []string{"email", "password"},
map[string][]string{"email": {"test@example.com"}})

    if err != nil {

        t.Fatalf("Ошибка при получении строки из таблицы users: %v", err)

    }

```

```

        // Проверяем, что данные совпадают с ожидаемыми

        if len(rows) != 1 || rows[0][0] != "test@example.com" || rows[0][1] !=
"password" {

            t.Errorf("Ожидались данные email=test@example.com,
password=password, но получены %v", rows)

        }

    }

// TestDeleteRows проверяет удаление строк из таблицы `users`.
func TestDeleteRows(t *testing.T) {

    dbManager := setupTestDBManager(t)

    defer dbManager.db.Close()

    // Добавляем тестового пользователя

    err := dbManager.AddRows("users", map[string][]string{"email":
{"delete_test@example.com"}, "password": {"password"}})

    if err != nil {

        t.Fatalf("Ошибка при добавлении тестового пользователя: %v",
err)

    }

    // Удаляем тестового пользователя

    err = dbManager.DeleteRows("users", map[string][]string{"email":
{"delete_test@example.com"}})

    if err != nil {

        t.Fatalf("Ошибка при удалении строки из таблицы users: %v", err)

```

```

    }

    // Проверяем, что данных больше нет

    rows, err := dbManager.GetRows("users", []string{"email"},
map[string][]string{"email": {"delete_test@example.com"}})

    if err != nil {

        t.Fatalf("Ошибка при выполнении запроса после удаления: %v",
err)

    }

    if len(rows) != 0 {

        t.Errorf("Ожидалась пустая выборка после удаления, но получены
данные: %v", rows)

    }

}

```

internal\db\dbTypes.go:

```
package db
```

```
import (
```

```
    "reflect"
```

```
)
```

```
type NamedEntity interface {
```

```

        GetValue() string
    }

func ValuesFromAny(entities interface{}) []string {
    v := reflect.ValueOf(entities)

    if v.Kind() != reflect.Slice {
        panic("expected a slice")
    }

    values := make([]string, v.Len())

    for i := 0; i < v.Len(); i++ {
        entity := v.Index(i).Interface().(NamedEntity)
        values[i] = entity.GetValue()
    }

    return values
}

```

internal\db\db_test.go:

```
package db
```



```
import (  
    "testing"  
)
```

```
const (  
    host    = "localhost"  
    port    = "5432"  
    user     = "postgres"  
    password = "1q2ws3edc4r"  
    dbname  = "DotaTest" // тестовая база данных  
)
```

// setupTestDB подключается к тестовой базе данных для использования в тестах.

```
func setupTestDB(t *testing.T) *DB {  
    t.Helper()  
    db, err := NewDB(host, port, user, password, dbname)  
    if err != nil {  
        t.Fatalf("Не удалось подключиться к тестовой базе данных: %v",  
err)  
    }  
    return db  
}
```

// TestNewDB_Success проверяет успешное подключение к базе данных.

```
func TestNewDB_Success(t *testing.T) {  
    db := setupTestDB(t)  
    defer db.Close()  
}
```

// TestNewDB_InvalidCredentials проверяет, что при неверных данных подключения функция возвращает ошибку.

```
func TestNewDB_InvalidCredentials(t *testing.T) {  
    _, err := NewDB(host, port, "wrong_user", "wrong_password", dbname)  
    if err == nil {  
        t.Error("Ожидалась ошибка при подключении с неверными  
учетными данными, но её не произошло")  
    }  
}
```

// TestDB_AddUser проверяет добавление пользователя в таблицу `users`.

```
func TestDB_AddUser(t *testing.T) {  
    db := setupTestDB(t)  
    defer db.Close()  
  
    // Добавление пользователя в таблицу `users`.  
    _, err := db.Exec("INSERT INTO users (email, password) VALUES ($1, $2)",  
"test@example.com", "password")
```

```

    if err != nil {

        t.Fatalf("Ошибка выполнения запроса INSERT: %v", err)

    }

    // Удаление добавленного пользователя для чистоты тестов.

    _, err = db.Exec("DELETE FROM users WHERE email = $1",
"test@example.com")

    if err != nil {

        t.Errorf("Ошибка удаления тестового пользователя: %v", err)

    }

}

// TestDB_GetUser проверяет выборку данных из таблицы `users`.

func TestDB_GetUser(t *testing.T) {

    db := setupTestDB(t)

    defer db.Close()

    // Добавляем тестового пользователя.

    _, err := db.Exec("INSERT INTO users (email, password) VALUES ($1, $2)",
"test@example.com", "password")

    if err != nil {

        t.Fatalf("Ошибка добавления тестового пользователя: %v", err)

    }

    defer db.Exec("DELETE FROM users WHERE email = $1",
"test@example.com") // Удаляем пользователя после теста.

```

```

// Выполнение запроса SELECT.

rows, err := db.Query("SELECT email, password FROM users WHERE email =
$1", "test@example.com")

if err != nil {

    t.Fatalf("Ошибка выполнения запроса SELECT: %v", err)

}

defer rows.Close()


// Проверяем, что результаты получены.

if !rows.Next() {

    t.Error("Ожидался хотя бы один результат, но строки
отсутствуют")

}


// Проверяем значения полей email и password.

var email, password string

if err := rows.Scan(&email, &password); err != nil {

    t.Errorf("Ошибка при чтении результата: %v", err)

}

if email != "test@example.com" || password != "password" {

    t.Errorf("Ожидались email 'test@example.com' и password
'password', но получены %s и %s", email, password)

}

}

```

// TestDB_Close проверяет корректное закрытие соединения с базой данных.

```
func TestDB_Close(t *testing.T) {
```

```
    db := setupTestDB(t)
```

```
    // Закрытие подключения.
```

```
    err := db.Close()
```

```
    if err != nil {
```

```
        t.Errorf("Ошибка при закрытии подключения: %v", err)
```

```
    }
```

```
    // Проверка, что соединение закрыто.
```

```
    if err := db.conn.Ping(); err == nil {
```

```
        t.Error("Ожидалась ошибка при пинге закрытого подключения, но  
соединение активно")
```

```
    }
```

```
}
```

internal\home\homeHandler.go:

```
package home
```

```
import (
```

```
    "OracleGo/internal/net"
```

```
    _ "fmt"
    "net/http"
)

func HomeHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]interface{}{}

    net.RenderTemplate(w, r, "home.html", data)
}
```

internal\ml\mlManager.go:

```
package ml
```

internal\ml\predictionHandler.go:

```
package ml
```

```
import (
```

```
    "OracleGo/internal/net"
```

```
    "OracleGo/internal/statistics"
```

```
    "encoding/json"
```

```
    "fmt"
```

```
"net/http"

"os"

"os/exec"

"strings"

)

var (

    dbTeamManager *statistics.TeamsDatabaseManager

    dbHeroManager *statistics.HeroesDatabaseManager

    dbPlayerManager *statistics.PlayersDatabaseManager

)

func init() {

    var err error

    // Инициализация менеджеров баз данных один раз

    dbTeamManager, err = statistics.NewTeamsDatabaseManager()

    if err != nil {

        fmt.Printf("Failed to create teams database manager: %v\n", err)

        return

    }

    dbHeroManager, err = statistics.NewHeroesDatabaseManager()

    if err != nil {

        fmt.Printf("Failed to create heroes database manager: %v\n", err)
```

```

        return
    }

    dbPlayerManager, err = statistics.NewPlayersDatabaseManager()

    if err != nil {
        fmt.Printf("Failed to create players database manager: %v\n", err)
        return
    }
}

func PredictionHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]interface{}{}

    if r.Method == http.MethodPost {
        if err := r.ParseForm(); err != nil {
            http.Error(w, "Unable to parse form", http.StatusBadRequest)
            return
        }

        team1 := r.FormValue("team1")
        team2 := r.FormValue("team2")

        var team1Players, team2Players []string

        teamRosters, err := dbTeamManager.GetTeamsRoastersList()

```



```

        if err != nil {

            http.Error(w, "Failed to get teams rosters list",
http.StatusInternalServerError)

            return

        }

team1Processed := false

team2Processed := false

for _, roster := range teamRosters {

    teamName := roster.Team.GetValue()

    if !team1Processed && teamName == team1 {

        for _, player := range roster.Players {

            team1Players = append(team1Players,
player.GetValue())

        }

        team1Processed = true

    } else if !team2Processed && teamName == team2 {

        for _, player := range roster.Players {

            team2Players = append(team2Players,
player.GetValue())

        }

        team2Processed = true

    }

```

```
        if team1Processed && team2Processed {  
            break  
        }  
    }  
}
```

```
team1Heroes := []string{  
    r.FormValue("team1-hero1"),  
    r.FormValue("team1-hero2"),  
    r.FormValue("team1-hero3"),  
    r.FormValue("team1-hero4"),  
    r.FormValue("team1-hero5"),  
}
```

```
team2Heroes := []string{  
    r.FormValue("team2-hero1"),  
    r.FormValue("team2-hero2"),  
    r.FormValue("team2-hero3"),  
    r.FormValue("team2-hero4"),  
    r.FormValue("team2-hero5"),  
}
```

```
allHeroes := append(team1Heroes, team2Heroes...)
```

```
var heroes []statistics.Hero
```

```

    for _, heroName := range allHeroes {
        heroes = append(heroes, statistics.Hero{Value: heroName})
    }

    heroWinrates, err := dbHeroManager.GetHeroesWinrates(heroes)

    if err != nil {
        http.Error(w, "Failed to get heroes winrates",
http.StatusInternalServerError)

        return
    }

    var team1HeroObjs, team2HeroObjs []statistics.Hero

    for _, name := range team1Heroes {
        team1HeroObjs = append(team1HeroObjs,
statistics.Hero{Value: name})
    }

    for _, name := range team2Heroes {
        team2HeroObjs = append(team2HeroObjs,
statistics.Hero{Value: name})
    }

    heroCounterPicks, err :=
dbHeroManager.GetHeroesCounterPicks(team1HeroObjs, team2HeroObjs)

    if err != nil {
        http.Error(w, "Failed to get heroes counter picks",
http.StatusInternalServerError)

        return
    }

```

```

var players []statistics.Player

for _, playerName := range append(team1Players, team2Players...) {
    players = append(players, statistics.Player{Value: playerName})
}

playerWinrates, err :=
dbPlayerManager.GetPlayerOnHeroWinrate(players, heroes)

if err != nil {
    http.Error(w, "Failed to get player winrates on heroes",
http.StatusInternalServerError)
    return
}

playerGameCounts, err :=
dbPlayerManager.GetPlayerCountOnHero(players, heroes)

if err != nil {
    http.Error(w, "Failed to get player game counts on heroes",
http.StatusInternalServerError)
    return
}

var rowData []string

rowData = append(rowData, team1, team2)

rowData = append(rowData, team1Players...)

rowData = append(rowData, team2Players...)

```

```

rowData = append(rowData, team1Heroes...)

rowData = append(rowData, team2Heroes...)


for _, winrate := range heroWinrates {
    rowData = append(rowData, fmt.Sprintf("%.2f",
winrate.Winrate))
}


for _, team1Hero := range team1Heroes {
    for _, team2Hero := range team2Heroes {
        found := false

        for _, counter := range heroCounterPicks {
            if counter.FirstHero.Value == team1Hero &&
counter.SecondHero.Value == team2Hero {
                rowData = append(rowData,
fmt.Sprintf("%.2f", counter.CounterPick))

                found = true
                break
            }
        }

        if !found {
            rowData = append(rowData, "0.00")
        }
    }
}

```

```

    for i, player := range team1Players {
        hero := team1Heroes[i]
        found := false
        for _, winrate := range playerWinrates {
            if winrate.Player.Value == player && winrate.Hero.Value
== hero {
                rowData = append(rowData, fmt.Sprintf("%.2f",
winrate.Winrate))
                found = true
                break
            }
        }
        if !found {
            rowData = append(rowData, "0.00")
        }
    }

```

```

    for i, player := range team2Players {
        hero := team2Heroes[i]
        found := false
        for _, winrate := range playerWinrates {
            if winrate.Player.Value == player && winrate.Hero.Value
== hero {

```



```

    }
}

for i, player := range team2Players {
    hero := team2Heroes[i]
    found := false
    for _, gameCount := range playerGameCounts {
        if gameCount.Player.Value == player &&
gameCount.Hero.Value == hero {
            rowData = append(rowData, fmt.Sprintf("%d",
gameCount.Count))
            found = true
            break
        }
    }
    if !found {
        rowData = append(rowData, "0")
    }
}

outputRow := strings.Join(rowData, ",")

if err := os.WriteFile("internal/ml/scripts/row.txt",
[]byte(outputRow), 0644); err != nil {

```



```
        http.Error(w, "Failed to write to row.txt",
http.StatusInternalServerError)
```

```
        return
```

```
    }
```

```
        cmd := exec.Command("python",
"internal/ml/scripts/run_model.py")
```

```
        output, err := cmd.CombinedOutput()
```

```
        if err != nil {
```

```
            http.Error(w, "Failed to run model script",
http.StatusInternalServerError)
```

```
            fmt.Printf("Script error: %s\n", err)
```

```
            fmt.Printf("Script output: %s\n", output)
```

```
            return
```

```
        }
```

```
        file2, err := os.Open("internal/ml/scripts/prediction_result.json")
```

```
        if err != nil {
```

```
            http.Error(w, "Failed to open prediction result file",
http.StatusInternalServerError)
```

```
            return
```

```
        }
```

```
        defer file2.Close()
```

```
        var resultData map[string]interface{}
```

```

        decoder := json.NewDecoder(file2)

        if err := decoder.Decode(&resultData); err != nil {

            http.Error(w, "Failed to parse prediction result",
http.StatusInternalServerError)

            return

        }

        data["PredictionResult"] = resultData["prediction"]
        data["PredictionProbability"] = resultData["probability"]
    }

    if teams, err := dbTeamManager.GetTeamsList(); err == nil {

        data["Team"] = teams

    }

    if heroList, err := dbHeroManager.GetHeroesNameList(); err == nil {

        data["Hero"] = heroList

    }

    net.RenderTemplate(w, r, "prediction.html", data)
}

```

internal\ml\recommendationHandler.go:

```
package ml
```

```

import (

    "OracleGo/internal/net"

    _ "fmt"

    "net/http"

)

func RecommendationsHandler(w http.ResponseWriter, r *http.Request) {

    data := map[string]interface{}{}

    net.RenderTemplate(w, r, "prediction.html", data)

}

```

internal\ml\scripts\prediction_result.json:

```
{"prediction": true, "probability": 0.6607487797737122}
```

internal\ml\scripts\row.txt:

```
nouns,Shopify Rebellion,Yuma,Copy,Gunnar,Lelis,Fly,Arteezy,Yopaj-,SabeRLight-
,Thiolicor,Kitrak,Abaddon,Ancient Apparition,Clockwerk,Chaos
Knight,Broodmother,Alchemist,Clinkz,Chen,Centaur
Warrunner,Bloodseeker,55.27,46.43,50.47,48.47,45.81,52.41,49.73,48.14,51.66,4
9.16,1.42,-1.44,1.38,0.12,-0.58,0.00,0.00,0.00,0.00,0.00,-0.17,0.41,1.46,-0.38,-
2.00,0.00,0.00,0.00,0.00,0.00,2.90,-2.72,-2.23,-
0.75,1.43,0.00,0.00,50.62,0.00,83.33,49.06,0.00,0.00,0.00,0.00,0,0,81,0,6,53,0,0,0
,0
```

internal\ml\scripts\run_model.py:

```
import os

import sys

import warnings

import numpy as np

import pandas as pd

import torch

from lightautoml.automl.presets.tabular_presets import TabularAutoML

import io

import pickle

import json


# Suppress all warnings and redirect stderr
sys.stderr = open(os.devnull, 'w')


N_THREADS = 4

N_FOLDS = 5

RANDOM_STATE = 42

TEST_SIZE = 0.2

TIMEOUT = 300

TARGET_NAME = '1'


def main():

    np.random.seed(RANDOM_STATE)
```

```
torch.set_num_threads(N_THREADS)
```

```
# Load model
```

```
with open("internal/ml/models/model.pkl", "rb") as file:
```

```
    model = pickle.load(file)
```

```
# Load data row
```

```
with open("internal/ml/scripts/row.txt", "r") as file:
```

```
    data = file.readline().strip()
```

```
# Prepare DataFrame
```

```
df = pd.read_csv(io.StringIO(data), header=None)
```

```
missing_values = [1, 2041, 7.31]
```

```
missing_df = pd.DataFrame([missing_values])
```

```
df = pd.concat([missing_df, df], axis=1).reset_index(drop=True)
```

```
df.columns = pd.Index([str(i) for i in range(1, 91)])
```

```
# Predict
```

```
result = model.predict(df)
```

```
# Extract prediction
```

```
prediction = bool(result.data[0][0] > 0.5)
```

```
probability = float(result.data[0][0])
```

```
# Prepare output and save to JSON file
```

```
output = {  
    "prediction": prediction,  
    "probability": probability  
}
```

```
# Save result to a JSON file
```

```
with open("internal/ml/scripts/prediction_result.json", "w") as json_file:  
    json.dump(output, json_file)
```

```
if __name__ == "__main__":  
    main()
```

```
internal\net\renderPage.go:
```

```
package net
```

```
import (  
    "html/template"  
    "log"  
    "net/http"
```

```
"os"

"path/filepath"

"github.com/gorilla/sessions"

)

var (
    store    = sessions.NewCookieStore([]byte("my-secret-key"))
    templates *template.Template
)

func GetTemplatePath() string {
    cwd, err := os.Getwd()
    if err != nil {
        log.Fatal(err)
    }

    for {
        templatePath := filepath.Join(cwd, "web", "templates")

        if _, err := os.Stat(templatePath); !os.IsNotExist(err) {
            return filepath.Join(templatePath, "*.html")
        }
    }
}
```

```

    parentDir := filepath.Dir(cwd)

    if parentDir == cwd {

        log.Fatalf("Не удалось найти папку с шаблонами, начальный
        путь был: %s", filepath.Join(cwd, "web", "templates", "*.html"))

    }

    cwd = parentDir

}
}

```

```

func init() {

    templatePath := GetTemplatePath()

    templates = template.Must(template.ParseGlob(templatePath))

}

```

```

func RenderTemplate(w http.ResponseWriter, r *http.Request, templateName
string, data map[string]interface{}) {

    session, _ := store.Get(r, "session-name")

    _, isLoggedIn := session.Values["email"]

    if data == nil {

        data = make(map[string]interface{})
    }
}

```



```

    }

    data["IsLoggedIn"] = isLoggedIn

    if err := templates.ExecuteTemplate(w, templateName, data); err != nil {

        http.Error(w, "Error rendering template",
http.StatusInternalServerError)

    }
}

```

internal\net\renderPage_test.go:

```
package net
```

```
import (
    "html/template"
    "net/http"
    "net/http/httptest"
    "os"
    "path/filepath"
    "strings"
    "testing"
)

```

// Функция для загрузки шаблонов в тестовом окружении

```
func loadTemplatesForTesting() {
```

```

    path := filepath.Join("../", "../", "web", "templates", "*.html")

    templates = template.Must(template.ParseGlob(path))
}

// Тест функции RenderTemplate, когда пользователь авторизован
func TestRenderTemplate_WithLoggedInUser(t *testing.T) {

    // Установка переменной окружения для тестовой базы шаблонов
    os.Setenv("GO_ENV", "test")

    defer os.Unsetenv("GO_ENV") // Очистка переменной окружения после
    теста

    // Загрузка шаблонов для тестовой среды
    loadTemplatesForTesting()

    // Создание тестового запроса и респондера
    req := httptest.NewRequest(http.MethodGet, "/", nil)
    w := httptest.NewRecorder()

    // Установка сессии с email для имитации авторизованного
    пользователя

    session, _ := store.Get(req, "session-name")

    session.Values["email"] = "test@example.com"

    session.Save(req, w)

```

```

// Вызов RenderTemplate с авторизованным пользователем

RenderTemplate(w, req, "example.html", map[string]interface{}{"Title":
"Test Title"})

resp := w.Result()

defer resp.Body.Close()

// Проверка, что статус ответа 200 OK

if resp.StatusCode != http.StatusOK {

    t.Errorf("Expected status 200, got %d", resp.StatusCode)

}

// Проверка, что тело ответа содержит сообщение для авторизованного
пользователя

body := w.Body.String()

if !strings.Contains(body, "Welcome, you are logged in!") {

    t.Errorf("Expected response to contain 'Welcome, you are logged in!',
but got %s", body)

}

}

// Тест функции RenderTemplate, когда пользователь не авторизован

func TestRenderTemplate_WithAnonymousUser(t *testing.T) {

    // Установка переменной окружения для тестовой базы шаблонов

    os.Setenv("GO_ENV", "test")

```

```
defer os.Unsetenv("GO_ENV") // Очистка переменной окружения после  
теста
```

```
// Загрузка шаблонов для тестовой среды
```

```
loadTemplatesForTesting()
```

```
// Создание тестового запроса и респондера
```

```
req := httptest.NewRequest(http.MethodGet, "/", nil)
```

```
w := httptest.NewRecorder()
```

```
// Вызов RenderTemplate с анонимным пользователем (без сессии)
```

```
RenderTemplate(w, req, "example.html", map[string]interface{}{"Title":  
"Test Title"})
```

```
resp := w.Result()
```

```
defer resp.Body.Close()
```

```
// Проверка, что статус ответа 200 OK
```

```
if resp.StatusCode != http.StatusOK {
```

```
    t.Errorf("Expected status 200, got %d", resp.StatusCode)
```

```
}
```

```
// Проверка, что тело ответа содержит сообщение для  
неавторизованного пользователя
```

```
body := w.Body.String()
```

```

        if !strings.Contains(body, "Please log in to continue.") {
            t.Errorf("Expected response to contain 'Please log in to continue.',
but got %s", body)
        }
    }
}

```

internal\net\session.go:

```
package net
```

```
import (
```

```
    "net/http"
```

```
    "github.com/gorilla/sessions"
```

```
)
```

```
func RedirectIfAuthenticated(next http.Handler) http.Handler {
```

```
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
```

```
        session, _ := store.Get(r, "session-name")
```

```
        if _, ok := session.Values["email"]; ok {
```

```
            http.Redirect(w, r, "/profile", http.StatusSeeOther)
```

```
            return
```

```
        }
```

```
        next.ServeHTTP(w, r)
```

```

    })
}

func SessionMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        session, _ := store.Get(r, "session-name")
        email := session.Values["email"]

        if email == nil {
            http.Redirect(w, r, "/login", http.StatusSeeOther)
            return
        }

        next.ServeHTTP(w, r)
    })
}

```

```

func SaveSession(w http.ResponseWriter, r *http.Request, email string) error {
    session, _ := store.Get(r, "session-name")
    session.Values["email"] = email
    session.Options = &sessions.Options{
        Path:    "/",
        MaxAge:  3600,
    }
}

```

```

        HttpOnly: true,
    }

    if err := session.Save(r, w); err != nil {
        http.Error(w, "Could not save session",
http.StatusInternalServerError)

        return err
    }

    return nil
}

```

internal\net\session_test.go:

```
package net
```

```
import (
    "net/http"
    "net/http/httptest"
    "testing"
)

```

```
func TestRedirectIfAuthenticated_RedirectsToProfile(t *testing.T) {
    // Создаем запрос и ответ

```

```
req := httptest.NewRequest(http.MethodGet, "/", nil)

w := httptest.NewRecorder()

// Устанавливаем сессию с email для имитации аутентифицированного
пользователя

session, _ := store.Get(req, "session-name")

session.Values["email"] = "test@example.com"

session.Save(req, w)

// Создаем тестовый обработчик, который вызывается, если не
происходит редирект

nextHandler := http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {

    w.WriteHeader(http.StatusOK)

})

handler := RedirectIfAuthenticated(nextHandler)

// Выполняем запрос

handler.ServeHTTP(w, req)

resp := w.Result()

// Проверяем, что произошел редирект на страницу профиля

if resp.StatusCode != http.StatusSeeOther {

    t.Errorf("Ожидался статус %v, но получен %v", http.StatusSeeOther,
resp.StatusCode)
```



```

    }

    if location := resp.Header.Get("Location"); location != "/profile" {
        t.Errorf("Ожидался редирект на /profile, но получен %v", location)
    }
}

```

```

func TestRedirectIfAuthenticated_AllowsAccessWhenUnauthenticated(t
*testing.T) {

    req := httptest.NewRequest(http.MethodGet, "/", nil)

    w := httptest.NewRecorder()

    // Создаем тестовый обработчик, который должен быть вызван

    nextHandler := http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {

        w.WriteHeader(http.StatusOK)

    })

    handler := RedirectIfAuthenticated(nextHandler)

    // Выполняем запрос

    handler.ServeHTTP(w, req)

    resp := w.Result()

    // Проверяем, что запрос прошел без редиректа

    if resp.StatusCode != http.StatusOK {

```

```
        t.Errorf("Ожидался статус %v, но получен %v", http.StatusOK,
resp.StatusCode)
    }
}
```

```
func TestSessionMiddleware_RedirectsToLoginWhenUnauthenticated(t
*testing.T) {

    req := httptest.NewRequest(http.MethodGet, "/protected", nil)

    w := httptest.NewRecorder()

    // Создаем тестовый обработчик

    nextHandler := http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {

        w.WriteHeader(http.StatusOK)

    })

    handler := SessionMiddleware(nextHandler)

    // Выполняем запрос

    handler.ServeHTTP(w, req)

    resp := w.Result()

    // Проверяем, что произошел редирект на страницу логина

    if resp.StatusCode != http.StatusSeeOther {

        t.Errorf("Ожидался статус %v, но получен %v", http.StatusSeeOther,
resp.StatusCode)
    }
}
```

```

    }

    if location := resp.Header.Get("Location"); location != "/login" {
        t.Errorf("Ожидался редирект на /login, но получен %v", location)
    }
}

func TestSessionMiddleware_AllowsAccessWhenAuthenticated(t *testing.T) {
    req := httptest.NewRequest(http.MethodGet, "/protected", nil)
    w := httptest.NewRecorder()

    // Устанавливаем сессию с email для имитации аутентифицированного
    // пользователя
    session, _ := store.Get(req, "session-name")
    session.Values["email"] = "test@example.com"
    session.Save(req, w)

    // Создаем тестовый обработчик, который должен быть вызван
    nextHandler := http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        w.WriteHeader(http.StatusOK)
    })

    handler := SessionMiddleware(nextHandler)

    // Выполняем запрос

```

```

handler.ServeHTTP(w, req)

resp := w.Result()

    // Проверяем, что доступ разрешен, поскольку пользователь
аутентифицирован

    if resp.StatusCode != http.StatusOK {

        t.Errorf("Ожидался статус %v, но получен %v", http.StatusOK,
resp.StatusCode)

    }

}

func TestSaveSession_SavesSessionSuccessfully(t *testing.T) {

    req := httptest.NewRequest(http.MethodGet, "/", nil)

    w := httptest.NewRecorder()

    // Сохраняем сессию с email

    err := SaveSession(w, req, "test@example.com")

    if err != nil {

        t.Fatalf("Ошибка сохранения сессии: %v", err)

    }

    // Проверяем, что сессия содержит email

    session, _ := store.Get(req, "session-name")

    email, ok := session.Values["email"].(string)

```

```
        if !ok || email != "test@example.com" {  
            t.Errorf("Ожидался email 'test@example.com' в сессии, но получен  
%v", email)  
        }  
    }  
}
```

internal\profile\profileHandler.go:

```
package profile
```

```
import (  
    "OracleGo/internal/net"  
    _ "fmt"  
    "net/http"  
)
```

```
func ProfileHandler(w http.ResponseWriter, r *http.Request) {  
    data := map[string]interface{}{}  
  
    net.RenderTemplate(w, r, "profile.html", data)  
}
```

internal\redis\redis.go:

```
package redis
```

```
import (
```

```
    "context"
```

```
    "encoding/json"
```

```
    "fmt"
```

```
    "log"
```

```
    "os"
```

```
    "time"
```

```
    "github.com/redis/go-redis/v9"
```

```
)
```

```
// RedisManager - структура для работы с Redis
```

```
type RedisManager struct {
```

```
    RedisHost  string
```

```
    RedisPort  string
```

```
    RedisPassword string
```

```
    redisClient *redis.Client
```

```
    ctx        context.Context
```

```
}
```

```
// NewRedisManager - конструктор для инициализации RedisManager
```

```

func NewRedisManager() *RedisManager {

    manager := &RedisManager{

        RedisHost:  getEnv("REDIS_HOST", "localhost"),

        RedisPort:   getEnv("REDIS_PORT", "6379"),

        RedisPassword: getEnv("REDIS_PASSWORD", ""),

        ctx:         context.Background(),

    }

    manager.redisClient = redis.NewClient(&redis.Options{

        Addr:  fmt.Sprintf("%s:%s", manager.RedisHost,
manager.RedisPort),

        Password: manager.RedisPassword, // пароль по умолчанию ""

        DB:  0,           // используемая БД по умолчанию

    })

    // Проверка подключения

    if err := manager.Ping(); err != nil {

        log.Fatalf("Не удалось подключиться к Redis: %v", err)

    }

    /*

    err := manager.redisClient.FlushDB(manager.ctx).Err()

    if err != nil {

        log.Fatalf("Ошибка при очистке базы данных: %v", err)

    }

```

```
*/  
  
return manager  
}
```

// Ping - проверка подключения к Redis

```
func (r *RedisManager) Ping() error {  
    _, err := r.redisClient.Ping(r.ctx).Result()  
    return err  
}
```

// CacheData - функция для кэширования данных

```
func (r *RedisManager) CacheData(key string, value interface{}, expiration  
time.Duration) error {  
    data, err := json.Marshal(value)  
    if err != nil {  
        return fmt.Errorf("ошибка при сериализации данных: %v", err)  
    }  
  
    err = r.redisClient.Set(r.ctx, key, data, expiration).Err()  
    if err != nil {  
        return fmt.Errorf("ошибка при сохранении данных в Redis: %v",  
err)  
    }  
}
```



```

        return nil
    }

    // GetCachedData - функция для получения кэшированных данных
    func (r *RedisManager) GetCachedData(key string, dest interface{}) error {
        data, err := r.redisClient.Get(r.ctx, key).Result()

        if err == redis.Nil {
            return fmt.Errorf("данные по ключу '%s' не найдены", key)
        } else if err != nil {
            return fmt.Errorf("ошибка при получении данных из Redis: %v",
err)
        }

        err = json.Unmarshal([]byte(data), dest)

        if err != nil {
            return fmt.Errorf("ошибка при десериализации данных: %v", err)
        }

        return nil
    }

```

// Вспомогательная функция для получения переменных окружения с дефолтным значением

```

func getEnv(key, defaultValue string) string {

```

```
    value, exists := os.LookupEnv(key)

    if !exists {

        return defaultValue

    }

    return value
}
```

internal\redis\redis_test.go:

```
package redis
```

```
import (

    "os"

    "testing"

    "time"

)
```

```
// Тест для инициализации RedisManager
```

```
func TestNewRedisManager(t *testing.T) {

    // Устанавливаем переменные окружения для тестирования

    os.Setenv("REDIS_HOST", "localhost")

    os.Setenv("REDIS_PORT", "6379")

    os.Setenv("REDIS_PASSWORD", "")
}
```

```

manager := NewRedisManager()

if err := manager.Ping(); err != nil {
    t.Fatalf("Ошибка подключения к Redis: %v", err)
}
}

// Тест для функции CacheData и GetCachedData
func TestCacheDataAndGetCachedData(t *testing.T) {
    manager := NewRedisManager()

    // Тестовые данные
    key := "testKey"
    value := map[string]string{"name": "Dota", "type": "Game"}
    expiration := 10 * time.Second

    // Сохраняем данные в кеш
    err := manager.CacheData(key, value, expiration)
    if err != nil {
        t.Fatalf("Ошибка кэширования данных: %v", err)
    }
}

```

```

// Получаем данные из кеша

var cachedValue map[string]string

err = manager.GetCachedData(key, &cachedValue)

if err != nil {

    t.Fatalf("Ошибка получения данных из кеша: %v", err)

}

// Проверяем, что значения совпадают

if cachedValue["name"] != value["name"] || cachedValue["type"] !=
value["type"] {

    t.Fatalf("Полученные данные не совпадают с исходными.
Ожидалось: %v, Получено: %v", value, cachedValue)

}

}

// Тест на получение данных, когда ключа нет

func TestGetCachedDataNotFound(t *testing.T) {

    manager := NewRedisManager()

    var cachedValue map[string]string

    err := manager.GetCachedData("nonexistentKey", &cachedValue)

    if err == nil {

        t.Fatalf("Ожидалась ошибка при получении несуществующего
ключа")

    }

}

```

```
}
```

```
// Тест для очистки кеша
```

```
func TestFlushDB(t *testing.T) {
```

```
    manager := NewRedisManager()
```

```
    // Устанавливаем тестовый ключ
```

```
    key := "flushTestKey"
```

```
    value := "someData"
```

```
    err := manager.CacheData(key, value, 5*time.Minute)
```

```
    if err != nil {
```

```
        t.Fatalf("Ошибка при кэшировании данных перед очисткой: %v",  
err)
```

```
    }
```

```
    // Очищаем базу данных
```

```
    err = manager.redisClient.FlushDB(manager.ctx).Err()
```

```
    if err != nil {
```

```
        t.Fatalf("Ошибка при очистке базы данных: %v", err)
```

```
    }
```

```
    // Проверяем, что данные удалены
```

```
    var cachedValue string
```

```
    err = manager.GetCachedData(key, &cachedValue)
```

```
        if err == nil {  
            t.Fatalf("Ожидалась ошибка, т.к. данные должны были быть  
удалены")  
        }  
    }  
}
```

internal\statistics\dbHeroes.go:

```
package statistics
```

```
import (  
    "OracleGo/internal/db"  
    "fmt"  
    _ "fmt"  
    "strconv"  
    "strings"  
  
    _ "github.com/lib/pq"  
)
```

```
type Hero struct {  
    Value string  
}
```

```
func (h Hero) GetValue() string {  
    return h.Value  
}
```

```
type Winrate struct {  
    Hero    Hero  
    Winrate float64  
}
```

```
type CounterRate struct {  
    FirstHero  Hero  
    SecondHero Hero  
    CounterPick float64  
}
```

```
type HeroesDatabaseManager struct {  
    dbManager db.DatabaseManager  
}
```

```
func NewHeroesDatabaseManager() (*HeroesDatabaseManager, error) {  
    dbManager, err := db.NewDatabaseManager()  
    if err != nil {  
        return nil, err  
    }  
}
```

```

    }

    return &HeroesDatabaseManager{dbManager: *dbManager}, nil
}

```

```

func (heroesDbManager *HeroesDatabaseManager) GetHeroesNameList()
([]Hero, error) {

    return heroesDbManager.GetHeroesNameListPerPatch("7.35c")

}

```

```

func (heroesDbManager *HeroesDatabaseManager)
getHeroesNameListPerPatchAsync(patch string, resultChan chan<- []Hero,
errorChan chan<- error) {

    heroesRows, err := heroesDbManager.dbManager.GetRows("heroes_list",
[]string{"hero_name"}, map[string][]string{"patch": {patch}})

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

```

```

    var result []Hero

    for _, row := range heroesRows {

        result = append(result, Hero{row[0]})

    }

```

```

    resultChan <- result

```



```
}
```

```
func (heroesDbManager *HeroesDatabaseManager)
GetHeroesNameListPerPatch(patch string) ([]Hero, error) {

    resultChan := make(chan []Hero)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов

    go heroesDbManager.getHeroesNameListPerPatchAsync(patch, resultChan,
errorChan)

    // Ожидание результата или ошибки

    select {

    case result := <-resultChan:

        return result, nil

    case err := <-errorChan:

        return nil, err

    }

}
```

```
func (heroesDbManager *HeroesDatabaseManager) GetAllHeroesWinrates()
([]Winrate, error) {

    return heroesDbManager.GetHeroesWinrates(make([]Hero, 0))

}
```

```

func (heroesDbManager *HeroesDatabaseManager)
getHeroesWinratesAsync(heroes []Hero, resultChan chan<- []Winrate, errorChan
chan<- error) {

    heroesNames := db.ValuesFromAny(heroes)

    params := map[string][]string{"patch": {"7.35c"}}

    if len(heroesNames) != 0 {

        params["hero_name"] = heroesNames

    }

    winratesRows, err := heroesDbManager.dbManager.GetRows("heroes_list",
[]string{"winrate", "hero_name"}, params)

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

    var result []Winrate

    for _, row := range winratesRows {

        winrate, convErr := strconv.ParseFloat(row[0], 64)

        if convErr != nil {

            errorChan <- fmt.Errorf("не удалось конвертировать string to
float: %w", convErr)

            return

        }

        result = append(result, Winrate{Winrate: winrate, Hero:
Hero{row[1]}})

```

```
}
```

```
resultChan <- result
```

```
}
```

```
func (heroesDbManager *HeroesDatabaseManager) GetHeroesWinrates(heroes  
[]Hero) ([]Winrate, error) {
```

```
    resultChan := make(chan []Winrate)
```

```
    errorChan := make(chan error)
```

```
    // Запуск асинхронной функции с передачей каналов
```

```
    go heroesDbManager.getHeroesWinratesAsync(heroes, resultChan,  
errorChan)
```

```
    // Ожидание результата или ошибки
```

```
    select {
```

```
    case result := <-resultChan:
```

```
        return result, nil
```

```
    case err := <-errorChan:
```

```
        return nil, err
```

```
    }
```

```
}
```

```

func (heroesDbManager *HeroesDatabaseManager)
getHeroesCounterPicksAsync(firstHeroes []Hero, secondHeroes []Hero,
resultChan chan<- []CounterRate, errorChan chan<- error) {

    firstHeroesNames := db.ValuesFromAny(firstHeroes)

    secondHeroesNames := db.ValuesFromAny(secondHeroes)

    var result []CounterRate

    for _, firstHero := range firstHeroesNames {

        multiFirstHero := strings.Split(strings.Repeat(firstHero+" ",
len(secondHeroes)), " ")[1:len(secondHeroes)]

        heroesRows, err :=
heroesDbManager.dbManager.GetRows("heroes_counters",
[]string{"counterrate", "first_hero_name", "second_hero_name"},
map[string][]string{"first_hero_name": multiFirstHero, "second_hero_name":
secondHeroesNames})

        if err != nil {

            errorChan <- fmt.Errorf("не удалось провести запрос: %w",
err)

            return

        }

        for _, row := range heroesRows {

            counterrate, convErr := strconv.ParseFloat(row[0], 64)

            if convErr != nil {

                errorChan <- fmt.Errorf("не удалось конвертировать
string to float: %w", convErr)

                return
            }
        }
    }
}

```

```

    }

    result = append(result, CounterRate{CounterPick: counterrate,
FirstHero: Hero{row[1]}, SecondHero: Hero{row[2]}})

    }

}

resultChan <- result

}

```

```

func (heroesDbManager *HeroesDatabaseManager)
GetHeroesCounterPicks(firstHeroes []Hero, secondHeroes []Hero) ([]CounterRate,
error) {

    resultChan := make(chan []CounterRate)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов

    go heroesDbManager.getHeroesCounterPicksAsync(firstHeroes,
secondHeroes, resultChan, errorChan)

    // Ожидание результата или ошибки

    select {

    case result := <-resultChan:

        return result, nil

    case err := <-errorChan:

        return nil, err
    }
}

```

```
    }  
}
```

internal\statistics\dbHeroes_test.go:

```
package statistics
```

```
import (  
    "OracleGo/internal/db"  
    "fmt"  
    "testing"  
)
```

// Функция для настройки тестового окружения с очисткой таблицы перед
каждым тестом

```
func setupTestHeroesDatabaseManager() (*HeroesDatabaseManager, error) {  
    dbManager, err := db.NewDatabaseManager()  
    if err != nil {  
        return nil, fmt.Errorf("failed to setup test database manager: %v",  
err)  
    }  
  
    return &HeroesDatabaseManager{dbManager: *dbManager}, nil  
}
```

```

func TestGetHeroesNameListPerPatch(t *testing.T) {

    manager, err := setupTestHeroesDatabaseManager()

    if err != nil {

        t.Fatalf("Failed to set up test database manager: %v", err)

    }

    // Выполнение теста (без вставки данных)

    heroes, err := manager.GetHeroesNameListPerPatch("7.35c")

    if err != nil {

        t.Errorf("GetHeroesNameListPerPatch returned an error: %v", err)

        return

    }

    expectedHeroes := []Hero{{Value: "Anti-Mage"}, {Value: "Axe"}}

    if len(heroes) != len(expectedHeroes) {

        t.Errorf("Expected %d heroes, got %d", len(expectedHeroes),
len(heroes))

    }

    for i, hero := range heroes {

        if hero.Value != expectedHeroes[i].Value {

            t.Errorf("Expected hero %s, got %s", expectedHeroes[i].Value,
hero.Value)

        }

    }

```

```

    }
}

func TestGetAllHeroesWinrates(t *testing.T) {
    manager, err := setupTestHeroesDatabaseManager()

    if err != nil {
        t.Fatalf("Failed to set up test database manager: %v", err)
    }

    // Выполнение теста (без вставки данных)

    winrates, err := manager.GetAllHeroesWinrates()

    if err != nil {
        t.Errorf("GetAllHeroesWinrates returned an error: %v", err)
        return
    }

    expectedWinrates := []Winrate{
        {Hero: Hero{Value: "Anti-Mage"}, Winrate: 53.2},
        {Hero: Hero{Value: "Axe"}, Winrate: 48.9},
    }

    if len(winrates) != len(expectedWinrates) {
        t.Errorf("Expected %d winrates, got %d", len(expectedWinrates),
len(winrates))
    }
}

```



```

    for i, winrate := range winrates {

        if winrate.Hero.Value != expectedWinrates[i].Hero.Value ||
winrate.Winrate != expectedWinrates[i].Winrate {

            t.Errorf("Expected winrate %+v, got %+v", expectedWinrates[i],
winrate)

        }

    }

}

```

```

func TestGetHeroesCounterPicks(t *testing.T) {

    manager, err := setupTestHeroesDatabaseManager()

    if err != nil {

        t.Fatalf("Failed to set up test database manager: %v", err)

    }

    // Выполнение теста (без вставки данных)

    firstHeroes := []Hero{{Value: "Anti-Mage"}}

    secondHeroes := []Hero{{Value: "Axe"}}

    counterPicks, err := manager.GetHeroesCounterPicks(firstHeroes,
secondHeroes)

    if err != nil {

        t.Errorf("GetHeroesCounterPicks returned an error: %v", err)

        return

    }
}

```

```

        expectedCounterPicks := []CounterRate{

            {FirstHero: Hero{Value: "Anti-Mage"}, SecondHero: Hero{Value:
"Axe"}, CounterPick: 55.1},

        }

        if len(counterPicks) != len(expectedCounterPicks) {

            t.Errorf("Expected %d counter picks, got %d",
len(expectedCounterPicks), len(counterPicks))

        }

        for i, counterPick := range counterPicks {

            if counterPick.FirstHero.Value !=
expectedCounterPicks[i].FirstHero.Value || counterPick.SecondHero.Value !=
expectedCounterPicks[i].SecondHero.Value || counterPick.CounterPick !=
expectedCounterPicks[i].CounterPick {

                t.Errorf("Expected counter pick %+v, got %+v",
expectedCounterPicks[i], counterPick)

            }

        }

    }
}

```

internal\statistics\dbPlayers.go:

```
package statistics
```

```
import (
```

```
    "OracleGo/internal/db"
```

```
    "fmt"  
    _ "fmt"  
    "strconv"  
  
    _ "github.com/lib/pq"  
)
```

```
type Player struct {  
    Value string  
}
```

```
func (p Player) GetValue() string {  
    return p.Value  
}
```

```
type PlayersDatabaseManager struct {  
    dbManager db.DatabaseManager  
}
```

```
type GamesCount struct {  
    Player Player  
    Hero  Hero  
    Count int
```

```
}
```

```
type PlayerWinrate struct {
```

```
    Player Player
```

```
    Hero  Hero
```

```
    Winrate float64
```

```
}
```

```
type PlayerWinrateCurPatch struct {
```

```
    Player Player
```

```
    Winrate float64
```

```
}
```

```
func NewPlayersDatabaseManager() (*PlayersDatabaseManager, error) {
```

```
    dbManager, err := db.NewDatabaseManager()
```

```
    if err != nil {
```

```
        return nil, err
```

```
    }
```

```
    return &PlayersDatabaseManager{dbManager: *dbManager}, nil
```

```
}
```

```
func (playersDbManager *PlayersDatabaseManager)
```

```
getPlayerOnHeroWinrateAsync(players []Player, heroes []Hero, resultChan chan<-  
[]PlayerWinrate, errorChan chan<- error) {
```

```

playerNames := db.ValuesFromAny(players)

heroesNames := db.ValuesFromAny(heroes)

winrateRows, err :=
playersDbManager.dbManager.GetRows("players_heroes_statistic",
[]string{"winrate", "player_name", "hero_name"},
map[string][]string{"player_name": playerNames, "hero_name": heroesNames})

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

var result []PlayerWinrate

for _, row := range winrateRows {

    winrate, convErr := strconv.ParseFloat(row[0], 64)

    if convErr != nil {

        errorChan <- fmt.Errorf("не удалось конвертировать string to
float: %w", convErr)

        return

    }

    result = append(result, PlayerWinrate{Winrate: winrate, Player:
Player{row[1]}, Hero: Hero{row[2]}})

}

resultChan <- result

}

```

```

func (playersDbManager *PlayersDatabaseManager)
GetPlayerOnHeroWinrate(players []Player, heroes []Hero) ([]PlayerWinrate, error)
{

    resultChan := make(chan []PlayerWinrate)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов

    go playersDbManager.getPlayerOnHeroWinrateAsync(players, heroes,
resultChan, errorChan)


    // Ожидание результата или ошибки

    select {

    case result := <-resultChan:

        return result, nil

    case err := <-errorChan:

        return nil, err

    }

}

```

```

func (playersDbManager *PlayersDatabaseManager)
getPlayerWinrateAsync(players []Player, resultChan chan<-
[]PlayerWinrateCurPatch, errorChan chan<- error) {

    playerNames := db.ValuesFromAny(players)

    winrateRows, err :=
playersDbManager.dbManager.GetRows("pro_players_list", []string{"winrate",
"player_name"}, map[string][]string{"player_name": playerNames, "patch":
{"7.35c"}})

```

```

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

    var result []PlayerWinrateCurPatch

    for _, row := range winrateRows {

        winrate, convErr := strconv.ParseFloat(row[0], 64)

        if convErr != nil {

            errorChan <- fmt.Errorf("не удалось конвертировать string to
float: %w", convErr)

            return

        }

        result = append(result, PlayerWinrateCurPatch{Winrate: winrate,
Player: Player{row[1]}})

    }

    resultChan <- result

}

```

```

func (playersDbManager *PlayersDatabaseManager) GetPlayersWinrate(players
[]Player) ([]PlayerWinrateCurPatch, error) {

    resultChan := make(chan []PlayerWinrateCurPatch)

    errorChan := make(chan error)

```

```

// Запуск асинхронной функции с передачей каналов

go playersDbManager.getPlayersWinrateAsync(players, resultChan,
errorChan)


// Ожидание результата или ошибки

select {

case result := <-resultChan:

    return result, nil

case err := <-errorChan:

    return nil, err

}

}


func (playersDbManager *PlayersDatabaseManager)
getPlayerCountOnHeroAsync(players []Player, heroes []Hero, resultChan chan<-
[]GamesCount, errorChan chan<- error) {

    playerNames := db.ValuesFromAny(players)

    heroesNames := db.ValuesFromAny(heroes)

    winrateRows, err :=
playersDbManager.dbManager.GetRows("players_heroes_statistic",
[]string{"count_of_matches", "player_name", "hero_name"},
map[string][]string{"player_name": playerNames, "hero_name": heroesNames})

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }
}

```



```

var result []GamesCount

for _, row := range winrateRows {

    count, convErr := strconv.Atoi(row[0])

    if convErr != nil {

        errorChan <- fmt.Errorf("не удалось конвертировать string to
int: %w", convErr)

        return

    }

    result = append(result, GamesCount{Count: count, Player:
Player{row[1]}, Hero: Hero{row[2]}})

}

resultChan <- result

}

```

```

func (playersDbManager *PlayersDatabaseManager)
GetPlayerCountOnHero(players []Player, heroes []Hero) ([]GamesCount, error) {

    resultChan := make(chan []GamesCount)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов

    go playersDbManager.getPlayerCountOnHeroAsync(players, heroes,
resultChan, errorChan)

```

```

        // Ожидание результата или ошибки

select {

    case result := <-resultChan:

        return result, nil

    case err := <-errorChan:

        return nil, err

}
}

```

internal\statistics\dbPlayers_test.go:

```
package statistics
```

```
import (

    "testing"

)

```

```

func TestGetPlayerOnHeroWinrate(t *testing.T) {

    // Инициализация менеджера базы данных

    dbManager, err := NewPlayersDatabaseManager()

    if err != nil {

        t.Fatalf("Не удалось инициализировать менеджер базы данных:
        %v", err)

    }
}

```

```

// Тестовые данные

players := []Player{{Value: "Ame"}}

heroes := []Hero{{Value: "Anti-Mage"}}


// Выполнение функции

result, err := dbManager.GetPlayerOnHeroWinrate(players, heroes)

if err != nil {
    t.Fatalf("Ошибка при получении винрейта игрока на героя: %v",
err)
}


// Проверка результатов

if len(result) == 0 {
    t.Error("Ожидался непустой результат")
} else {
    for _, winrate := range result {
        if winrate.Player.Value != "Ame" || winrate.Hero.Value !=
"Anti-Mage" || winrate.Winrate != 55.0 {
            t.Errorf("Неверный результат: ожидался {Player: 'Ame',
Hero: 'Anti-Mage', Winrate: 55}, получено %+v", winrate)
        }
    }
}
}

```

```

func TestGetPlayerCountOnHero(t *testing.T) {

    // Инициализация менеджера базы данных

    dbManager, err := NewPlayersDatabaseManager()

    if err != nil {

        t.Fatalf("Не удалось инициализировать менеджер базы данных:
%v", err)

    }


    // Тестовые данные

    players := []Player{{Value: "Ame"}}

    heroes := []Hero{{Value: "Anti-Mage"}}


    // Выполнение функции

    result, err := dbManager.GetPlayerCountOnHero(players, heroes)

    if err != nil {

        t.Fatalf("Ошибка при получении количества матчей игрока на
герое: %v", err)

    }


    // Проверка результатов

    if len(result) == 0 {

        t.Error("Ожидался непустой результат")

    } else {

        for _, count := range result {

```

```

        if count.Player.Value != "Ame" || count.Hero.Value != "Anti-
Mage" || count.Count != 100 {

            t.Errorf("Неверный результат: ожидался {Player: 'Ame',
Hero: 'Anti-Mage', Count: 100}, получено %+v", count)

        }

    }

}

}

```

internal\statistics\dbTeams.go:

```
package statistics
```

```
import (

    "OracleGo/internal/db"

    "fmt"

    _ "fmt"

    _ "github.com/lib/pq"

)

```

```
type Team struct {

    Value string

}

```

```
func (t Team) GetValue() string {  
    return t.Value  
}
```

```
type TeamRoaster struct {  
    Players []Player  
    Team    Team  
}
```

```
type TeamsDatabaseManager struct {  
    dbManager db.DatabaseManager  
}
```

```
func NewTeamsDatabaseManager() (*TeamsDatabaseManager, error) {  
    dbManager, err := db.NewDatabaseManager()  
    if err != nil {  
        return nil, err  
    }  
    return &TeamsDatabaseManager{dbManager: *dbManager}, nil  
}
```

```
func (teamsDbManager *TeamsDatabaseManager) getTeamsListAsync(resultChan  
chan<- []Team, errorChan chan<- error) {
```

```

        teamsRows, err :=
teamsDbManager.dbManager.GetRows("teams_roasters",
[]string{"team_name"}, map[string][]string{})

        if err != nil {

                errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

                return

        }

var result []Team

for _, row := range teamsRows {

        result = append(result, Team{row[0]})

}

// Отправка результата в канал

resultChan <- result

}

func (teamsDbManager *TeamsDatabaseManager) GetTeamsList() ([]Team, error)
{

        resultChan := make(chan []Team)

        errorChan := make(chan error)

        // Запуск асинхронной функции с передачей каналов

        go teamsDbManager.getTeamsListAsync(resultChan, errorChan)

```

```

// Ожидание результата или ошибки

select {

case result := <-resultChan:

    return result, nil

case err := <-errorChan:

    return nil, err

}

}

func (teamsDbManager *TeamsDatabaseManager)
getTeamsRoastersListAsync(resultChan chan<- []TeamRoaster, errorChan chan<-
error) {

    teamsRows, err :=
teamsDbManager.dbManager.GetRows("teams_roasters", []string{"*"},
map[string][]string{})

    if err != nil {

        errorChan <- fmt.Errorf("не удалось провести запрос: %w", err)

        return

    }

    var result []TeamRoaster

    for _, row := range teamsRows {

        result = append(result, TeamRoaster{

            Team: Team{Value: row[0]},

            Players: []Player{

```



```

        {row[1]}, {row[2]}, {row[3]}, {row[4]}, {row[5]},

        },

    })

}

// Отправка результата в канал
resultChan <- result

}

func (teamsDbManager *TeamsDatabaseManager) GetTeamsRoastersList()
([]TeamRoaster, error) {

    resultChan := make(chan []TeamRoaster)

    errorChan := make(chan error)

    // Запуск асинхронной функции с передачей каналов
    go teamsDbManager.getTeamsRoastersListAsync(resultChan, errorChan)

    // Ожидание результата или ошибки
    select {

    case result := <-resultChan:

        return result, nil

    case err := <-errorChan:

        return nil, err

    }
}

```

```
}
```

```
internal\statistics\dbTeams_test.go:
```

```
package statistics
```

```
import (
```

```
    "testing"
```

```
)
```

```
func TestGetTeamsList(t *testing.T) {
```

```
    // Инициализация менеджера базы данных команд
```

```
    teamsDbManager, err := NewTeamsDatabaseManager()
```

```
    if err != nil {
```

```
        t.Fatalf("Не удалось создать менеджер базы данных: %v", err)
```

```
    }
```

```
    // Получение списка команд
```

```
    teams, err := teamsDbManager.GetTeamsList()
```

```
    if err != nil {
```

```
        t.Fatalf("Ошибка при выполнении GetTeamsList: %v", err)
```

```
    }
```

```
// Проверка наличия тестовой команды

expectedTeam := "Nouns"

found := false

for _, team := range teams {

    if team.Value == expectedTeam {

        found = true

        break

    }

}

if !found {

    t.Errorf("Команда %s не найдена в результате", expectedTeam)

}

}
```

```
func TestGetTeamsRoastersList(t *testing.T) {

    // Инициализация менеджера базы данных команд

    teamsDbManager, err := NewTeamsDatabaseManager()

    if err != nil {

        t.Fatalf("Не удалось создать менеджер базы данных: %v", err)

    }

    // Получение списка составов команд
```

```

roasters, err := teamsDbManager.GetTeamsRoastersList()

if err != nil {

    t.Fatalf("Ошибка при выполнении GetTeamsRoastersList: %v", err)

}

// Проверка наличия тестового состава команды

expectedTeam := "Nouns"

expectedPlayers := []string{"A", "B", "C", "D", "E"}

found := false

for _, roaster := range roasters {

    if roaster.Team.Value == expectedTeam {

        found = true

        for i, player := range roaster.Players {

            if player.GetValue() != expectedPlayers[i] {

                t.Errorf("Ожидаемый игрок %s, но найден %s",
expectedPlayers[i], player.GetValue())

            }

        }

        break

    }

}

}

if !found {

```

```
                t.Errorf("Состав команды %s не найден в результате",
expectedTeam)
            }
        }
    }
```

internal\statistics\statisticsHandler.go:

```
package statistics
```

```
import (
    "OracleGo/internal/net"
    _ "fmt"
    "net/http"
)
```

```
func StatisticsHandler(w http.ResponseWriter, r *http.Request) {
    data := map[string]interface{}{}
    dbManager, _ := NewHeroesDatabaseManager()
    heroes, _ := dbManager.GetAllHeroesWinrates()
    data["HeroesWinrates"] = heroes
    net.RenderTemplate(w, r, "statistics.html", data)
}
```

internal\statistics\teamsHandler.go:

```
package statistics
```

```
import (
```

```
    "OracleGo/internal/net"
```

```
    _ "fmt"
```

```
    "net/http"
```

```
)
```

```
func TeamsHandler(w http.ResponseWriter, r *http.Request) {
```

```
    data := map[string]interface{}{}
```

```
    dbManager, _ := NewTeamsDatabaseManager()
```

```
    roasters, _ := dbManager.GetTeamsRoastersList()
```

```
    data["TeamsRoaters"] = roasters
```

```
    net.RenderTemplate(w, r, "teams.html", data)
```

```
}
```

web\static\style.css:

```
/* ОСНОВНЫЕ СТИЛИ */
```

```
header,
```

```
main,
```

```
footer {
```

```
width: 100%; /* Занимает всю ширину */

display: block; /* Размещает секции по вертикали */

margin: 0; /* Убирает отступы */

padding: 0;

}


body {

    font-family: 'Roboto', sans-serif;

    background-color: #1a1a1a;

    color: #f0f0f0;

    margin: 0;

    padding: 0;

    min-height: 100vh;

}


header {

    background: linear-gradient(90deg, #111, #333); /* Градиентная
навигация */

    padding: 15px;

    text-align: center;

    box-shadow: 0px 4px 15px rgba(0, 0, 0, 0.5); /* Тени для футуристичного
эффекта */

}
```

```
nav ul {  
    list-style: none;  
    padding: 0;  
    margin: 0;  
    display: flex;  
    justify-content: center;  
}
```

```
nav ul li {  
    margin: 0 20px;  
}
```

```
nav ul li a {  
    color: #e63946;  
    text-decoration: none;  
    font-weight: bold;  
    font-size: 16px;  
    transition: color 0.3s ease;  
}
```

```
nav ul li a:hover {  
    color: #f0f0f0; /* Меняем цвет при наведении */  
}
```



```
main {  
  
    max-width: 1200px;  
  
    margin: 50px auto;  
  
    padding: 30px;  
  
    background-color: #2c2c2c;  
  
    box-shadow: 0 2px 15px rgba(0, 0, 0, 0.6); /* Глубокая тень */  
  
    border-radius: 15px;  
  
    animation: fadeIn 1s ease-in; /* Анимация появления */  
  
}
```

```
/* Анимация плавного появления */
```

```
@keyframes fadeIn {  
  
    from {  
  
        opacity: 0;  
  
        transform: translateY(20px);  
  
    }  
  
    to {  
  
        opacity: 1;  
  
        transform: translateY(0);  
  
    }  
  
}
```

```
h1,  
h2,  
h3 {  
  
    color: #e63946; /* Заголовки с яркими красными акцентами */  
  
}
```

```
h1 {  
  
    font-size: 36px;  
  
}
```

```
h2 {  
  
    font-size: 30px;  
  
}
```

```
h3 {  
  
    font-size: 24px;  
  
}
```

```
p {  
  
    font-size: 18px;  
  
    line-height: 1.8;  
  
    color: #d1d1d1; /* Более светлый текст */  
  
}
```

```
.btn {  
  
    display: inline-block;  
  
    padding: 15px 25px;  
  
    background: linear-gradient(90deg, #e63946, #d62828);  
  
    color: #fff;  
  
    text-decoration: none;  
  
    border-radius: 10px;  
  
    font-weight: bold;  
  
    transition: background-color 0.3s ease, transform 0.3s ease;  
  
}
```

```
.btn:hover {  
  
    background: linear-gradient(90deg, #ff4f5e, #ff313a);  
  
    transform: scale(1.05); /* Легкая анимация увеличения */  
  
}
```

/* Статистические блоки */

```
.stats-container {  
  
    display: flex;  
  
    justify-content: space-around;  
  
    flex-wrap: wrap;  
  
    gap: 20px;
```

```
}
```

```
.stat-box {
```

```
    background-color: #1f1f1f;
```

```
    padding: 30px;
```

```
    text-align: center;
```

```
    border-radius: 10px;
```

```
    flex: 1;
```

```
    min-width: 250px;
```

```
    margin: 10px;
```

```
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.4);
```

```
    transition: transform 0.3s ease;
```

```
}
```

```
.stat-box:hover {
```

```
    transform: translateY(-5px); /* Плавное поднятие блока при наведении */
```

```
}
```

```
.stat-box h3 {
```

```
    font-size: 22px;
```

```
    color: #fff;
```

```
}
```

```
.stat-box p {  
  
    font-size: 20px;  
  
    color: #e63946; /* Красные акценты для текста */  
  
}
```

```
/* Таблицы */
```

```
table {  
  
    width: 100%;  
  
    border-collapse: collapse;  
  
    margin: 30px 0;  
  
    color: #d1d1d1;  
  
    background-color: #2a2a2a;  
  
}
```

```
table th,
```

```
table td {  
  
    padding: 15px;  
  
    text-align: left;  
  
    border-bottom: 1px solid #444;  
  
}
```

```
table th {  
  
    background-color: #e63946;
```

```
        color: #fff;
    }

table tr:hover {
    background-color: #3a3a3a; /* Легкий цветовой акцент при наведении */
}

/* Футер */
footer {
    background-color: #111;
    color: #f0f0f0;
    text-align: center;
    padding: 20px 0;
    box-shadow: 0 -2px 10px rgba(0, 0, 0, 0.5);
    margin-top: 30px; /* Добавляем отступ сверху, чтобы подвал не
сливался с контентом */
}

/* Стили выпадающих меню */
.dropdown {
    width: 100%;
    padding: 10px;
    border-radius: 5px;
    border: 1px solid #444;
    background-color: #2a2a2a;
```

```
    color: #f0f0f0;

    font-size: 16px;

    margin-bottom: 20px;
}
```

```
.dropdown option {

    background-color: #1a1a1a;
}
```

```
.prediction-form {

    margin-top: 30px;
}
```

```
.team-select {

    margin-bottom: 20px;
}
```

```
.hero-select-container {

    margin-top: 30px;
}
```

```
.hero-row {

    display: flex;
```

```
    justify-content: space-between;

    gap: 20px;

    margin-bottom: 20px;
}
```

```
.hero-select {

    flex: 1;
}
```

```
.hero-select label {

    display: block;

    margin-bottom: 10px;

    font-size: 16px;
}
```

```
.prediction-button {

    text-align: center;

    margin-top: 30px;
}
```

```
/* Заголовок страницы */
```

```
.page-header {

    background: linear-gradient(90deg, #111, #333); /* Градиентная
навигация */
```



```
padding: 15px;

text-align: center;

box-shadow: 0px 4px 15px rgba(0, 0, 0, 0.5); /* Тени для футуристичного
эффекта */
}
```

```
.page-header h2 {

color: #e63946;

font-size: 32px;

margin-top: 30px;

margin-bottom: 20px;

}
```

```
/* Контейнер формы */
```

```
.form-container {

width: 100%;

max-width: 400px;

background-color: #2e2e2e;

padding: 40px;

border-radius: 15px;

box-shadow: 0 2px 15px rgba(0, 0, 0, 0.6);

}
```

```
/* Группы полей формы */
```

```
.form-group {  
    margin-bottom: 20px;  
}
```

```
.form-group label {  
    display: block;  
    font-size: 14px;  
    color: #d1d1d1;  
    margin-bottom: 8px;  
}
```

```
.form-group input {  
    width: 100%;  
    padding: 12px;  
    font-size: 16px;  
    color: #f0f0f0;  
    background-color: #1a1a1a;  
    border: 1px solid #444;  
    border-radius: 8px;  
    transition: border 0.3s;  
}
```

```
.form-group input:focus {
```

```
border: 1px solid #e63946;

outline: none;

box-shadow: 0 0 8px rgba(230, 57, 70, 0.6);

}


/* Кнопка отправки */

.btn-submit {

    width: 100%;

    padding: 12px;

    font-size: 18px;

    font-weight: bold;

    color: #fff;

    background: linear-gradient(90deg, #e63946, #d62828);

    border: none;

    border-radius: 8px;

    cursor: pointer;

    transition: background 0.3s ease, transform 0.2s;

}


.btn-submit:hover {

    background: linear-gradient(90deg, #ff4f5e, #ff313a);

    transform: scale(1.05);

}
```

```
/* Альтернативный вариант */
```

```
.alt-option {  
    text-align: center;  
    color: #d1d1d1;  
    margin-top: 15px;  
}
```

```
.alt-option a {  
    color: #e63946;  
    text-decoration: none;  
    font-weight: bold;  
}
```

```
.alt-option a:hover {  
    text-decoration: underline;  
}
```

```
web\templates\home.html:
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
    <head>
```

```
<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-
scale=1.0" />

<title>Dota2 Match Prediction - Home</title>

<link rel="stylesheet" href="/web/static/style.css" />

</head>

<body>

    <header>

        <nav>

            <ul>

                <li><a href="/">Home</a></li>

                <li><a href="/prediction">Match
Prediction</a></li>

                <li><a href="/statistics">Statistics</a></li>

                <li><a href="/team-analysis">Team
Analysis</a></li>

                <li><a href="/recommendations">Hero
Recommendations</a></li>

                {{if .IsLoggedIn}}

                <!-- Кнопка профиля, если пользователь
авторизован -->

                <li><a class="btn-profile"
href="/profile">Profile</a></li>

                {{else}}

                <!-- Кнопки авторизации и регистрации, если
пользователь не авторизован -->
```

```
        <li><a class="btn-login"
href="/login">Login</a></li>
```

```
        <li><a class="btn-register"
href="/register">Register</a></li>
```

```
    {{end}}
```

```
</ul>
```

```
</nav>
```

```
</header>
```

```
<main>
```

```
    <section class="intro">
```

```
        <h1>Welcome to Dota2 Match Prediction</h1>
```

```
        <p>
```

```
            Our service predicts Dota2 match outcomes using
AI models. Check out a
```

```
            sample prediction of a recent popular match!
```

```
        </p>
```

```
    </section>
```

```
    <section class="sample-prediction">
```

```
        <h2>Recent Match Prediction</h2>
```

```
        <div class="prediction-result">
```

```
            <p><strong>Team A</strong> vs <strong>Team
B</strong></p>
```

```
        <p>
```

Predicted Winner: **Team A** with a probability of

68%

[Predict Your Match](/prediction)

Statistics

Matches Processed

1050

Prediction Accuracy

85%

```
Winrates</a></p>
</div>
</div>
</section>
</main>

<footer>
    <p>&copy; 2024 Dota2 Match Prediction</p>
</footer>

</body>
</html>
```

web\templates\login.html:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8" />
        <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
        <title>Login</title>
        <link rel="stylesheet" href="./web/static/style.css" />
```



```
</head>

<body>

    <header>

        <nav>

            <ul>

                <li><a href="/">Home</a></li>

                <li><a href="/prediction">Match
Prediction</a></li>

                <li><a href="/statistics">Statistics</a></li>

                <li><a href="/team-analysis">Team
Analysis</a></li>

                <li><a href="/recommendations">Hero
Recommendations</a></li>

                {{if .IsLoggedIn}}

                <li><a class="btn-profile"
href="/profile">Profile</a></li>

                {{else}}

                <li><a class="btn-login"
href="/login">Login</a></li>

                <li><a class="btn-register"
href="/register">Register</a></li>

                {{end}}

            </ul>

        </nav>

    </header>
```

```
<main>

    <section class="form-container">

        <form action="/login" method="POST">

            <label for="email">Email</label>

            <input type="email" id="email" name="email"
required />

            <label for="password">Password</label>

            <input type="password" id="password"
name="password" required />

            <button type="submit" class="btn-
submit">Login</button>

        </form>

        <p>Don't have an account? <a href="/register">Register
here</a></p>

        {{if .ErrorMessage}}

        <div class="error-message">{{.ErrorMessage}}</div>

        {{end}}

    </section>

</main>

</body>

</html>
```

web\templates\prediction.html:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
    <title>Match Prediction</title>
```

```
    <link rel="stylesheet" href="./web/static/style.css" />
```

```
</head>
```

```
<body>
```

```
    <header>
```

```
        <nav>
```

```
            <ul>
```

```
                <li><a href="/">Home</a></li>
```

```
                <li><a href="/prediction">Match Prediction</a></li>
```

```
                <li><a href="/statistics">Statistics</a></li>
```

```
                <li><a href="/team-analysis">Team Analysis</a></li>
```

```
                <li><a href="/recommendations">Hero Recommendations</a></li>
```

```
                {{if .IsLoggedIn}}
```

```
                <li><a class="btn-profile" href="/profile">Profile</a></li>
```

```
        {{else}}

        <li><a class="btn-login" href="/login">Login</a></li>

        <li><a class="btn-register" href="/register">Register</a></li>

        {{end}}

    </ul>

</nav>

</header>


<main>

    <section>

        <h1>Predict Match Outcome</h1>

        <p>Select the teams and heroes to get a prediction for the match
outcome.</p>


        {{if .PredictionResult}}

        <div class="prediction-result">

            <h2>Prediction Result</h2>

            <p>The predicted outcome is:
<strong>{{.PredictionResult}}</strong></p>


            {{if .PredictionProbability}}

            <p>Probability of this outcome:
<strong>{{.PredictionProbability}}</strong></p>

            {{end}}

        </div>

    </section>

</main>
```

{{end}}

<!-- Начало формы -->

<form action="/prediction" method="POST" class="prediction-form">

<!-- Команды -->

<div class="team-select">

<label for="team1">Team 1:</label>

<select id="team1" name="team1" class="dropdown">

<option value="" disabled selected>Select Team 1</option>

{{range .Team}}

<option value="{{.Value}}">{{.Value}}</option>

{{end}}

</select>

</div>

<div class="team-select">

<label for="team2">Team 2:</label>

<select id="team2" name="team2" class="dropdown">

<option value="" disabled selected>Select Team 2</option>

{{range .Team}}

<option value="{{.Value}}">{{.Value}}</option>

{{end}}

</select>

</div>

<!-- Герои -->

<div class="hero-select-container">

<h2>Pick Heroes</h2>

<div class="hero-row">

<div class="hero-select">

<label for="team1-hero1">Team 1 Hero 1:</label>

<select id="team1-hero1" name="team1-hero1"
class="dropdown">

<option value="" disabled selected>Select Hero 1</option>

{{range .Hero}}

<option value="{{.Value}}">{{.Value}}</option>

{{end}}

</select>

</div>

<div class="hero-select">

<label for="team2-hero1">Team 2 Hero 1:</label>

<select id="team2-hero1" name="team2-hero1"
class="dropdown">

<option value="" disabled selected>Select Hero 1</option>

{{range .Hero}}

```
        <option value="{{.Value}}">{{.Value}}</option>

        {{end}}

    </select>

</div>

</div>
```

<!-- Повторите для других героев -->

```
<div class="hero-row">

    <div class="hero-select">

        <label for="team1-hero2">Team 1 Hero 2:</label>

        <select id="team1-hero2" name="team1-hero2"
class="dropdown">

            <option value="" disabled selected>Select Hero 2</option>

            {{range .Hero}}

                <option value="{{.Value}}">{{.Value}}</option>

            {{end}}

        </select>

    </div>
```

```
<div class="hero-select">

    <label for="team2-hero2">Team 2 Hero 2:</label>

    <select id="team2-hero2" name="team2-hero2"
class="dropdown">

        <option value="" disabled selected>Select Hero 2</option>
```

```

        {{range .Hero}}

        <option value="{{.Value}}">{{.Value}}</option>

        {{end}}

    </select>

</div>

</div>

```

<!-- Добавьте все остальные герои здесь с уникальными атрибутами `name` -->

```

        <div class="hero-row">

        <div class="hero-select">

            <label for="team1-hero3">Team 1 Hero 3:</label>

            <select id="team1-hero3" name="team1-hero3"
class="dropdown">

                <option value="" disabled selected>Select Hero 3</option>

                {{range .Hero}}

                <option value="{{.Value}}">{{.Value}}</option>

                {{end}}

            </select>

        </div>

        <div class="hero-select">

            <label for="team2-hero3">Team 2 Hero 3:</label>

            <select id="team2-hero3" name="team2-hero3"
class="dropdown">

```



```
        <option value="" disabled selected>Select Hero 3</option>

        {{range .Hero}}

        <option value="{{.Value}}">{{.Value}}</option>

        {{end}}

    </select>

</div>

</div>
```

```
        <div class="hero-row">

        <div class="hero-select">

            <label for="team1-hero4">Team 1 Hero 4:</label>

            <select id="team1-hero4" name="team1-hero4"
class="dropdown">

                <option value="" disabled selected>Select Hero 4</option>

                {{range .Hero}}

                <option value="{{.Value}}">{{.Value}}</option>

                {{end}}

            </select>

        </div>
```

```
        <div class="hero-select">

            <label for="team2-hero4">Team 2 Hero 4:</label>

            <select id="team2-hero4" name="team2-hero4"
class="dropdown">
```

```
        <option value="" disabled selected>Select Hero 4</option>

        {{range .Hero}}

        <option value="{{.Value}}">{{.Value}}</option>

        {{end}}

    </select>

</div>

</div>
```

```
        <div class="hero-row">

        <div class="hero-select">

            <label for="team1-hero5">Team 1 Hero 5:</label>

            <select id="team1-hero5" name="team1-hero5"
class="dropdown">

                <option value="" disabled selected>Select Hero 5</option>

                {{range .Hero}}

                <option value="{{.Value}}">{{.Value}}</option>

                {{end}}

            </select>

        </div>
```

```
        <div class="hero-select">

            <label for="team2-hero5">Team 2 Hero 5:</label>

            <select id="team2-hero5" name="team2-hero5"
class="dropdown">
```

```
        <option value="" disabled selected>Select Hero 5</option>

        {{range .Hero}}

        <option value="{{.Value}}">{{.Value}}</option>

        {{end}}

    </select>

</div>

</div>

</div>

<!-- Кнопка для предсказания -->

<div class="prediction-button">

    <button type="submit" class="btn">Get Prediction</button>

</div>

</form>

<!-- Конец формы -->

</section>

</main>


<footer>

    <p>&copy; 2024 My Dota Site. All rights reserved.</p>

</footer>

</body>

</html>
```

web\templates\profile.html:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
    <head>
```

```
        <meta charset="UTF-8" />
```

```
        <meta name="viewport" content="width=device-width, initial-  
scale=1.0" />
```

```
        <title>User Profile</title>
```

```
        <link rel="stylesheet" href="/web/static/style.css" />
```

```
    </head>
```

```
    <body>
```

```
        <header>
```

```
            <nav>
```

```
                <ul>
```

```
                    <li><a href="/">Home</a></li>
```

```
                    <li><a href="/prediction">Match  
Prediction</a></li>
```

```
                    <li><a href="/statistics">Statistics</a></li>
```

```
                    <li><a href="/team-analysis">Team  
Analysis</a></li>
```

```
                    <li><a href="/recommendations">Hero  
Recommendations</a></li>
```

```
        {{if .IsLoggedIn}}

        <li><a class="btn-profile"
href="/profile">Profile</a></li>

        {{else}}

        <li><a class="btn-login"
href="/login">Login</a></li>

        <li><a class="btn-register"
href="/register">Register</a></li>

        {{end}}

    </ul>

</nav>

</header>

<main>

    <section class="profile">

        <h1>Your Profile</h1>

        <h2>Prediction History</h2>

        <ul>

            <li>Match 1 - Result: Correct</li>

            <li>Match 2 - Result: Incorrect</li>

            <!-- Добавь больше истории -->

        </ul>

    </section>

    <section class="personal-stats">
```

```
        <h2>Your Accuracy</h2>

        <p>Accuracy: 78%</p>

    </section>

</main>

<footer>

    <p>&copy; 2024 Dota2 Match Prediction</p>

</footer>

</body>

</html>
```

web\templates\recommendations.html:

```
<!DOCTYPE html>

<html lang="en">

    <head>

        <meta charset="UTF-8" />

        <meta name="viewport" content="width=device-width, initial-
scale=1.0" />

        <title>Hero Recommendations</title>

        <link rel="stylesheet" href="./web/static/style.css" />

    </head>

    <body>

        <header>
```

```
<nav>

    <ul>

        <li><a href="/">Home</a></li>

        <li><a href="/prediction">Match
Prediction</a></li>

        <li><a href="/statistics">Statistics</a></li>

        <li><a href="/team-analysis">Team
Analysis</a></li>

        <li><a href="/recommendations">Hero
Recommendations</a></li>

        {{if .IsLoggedIn}}

            <li><a class="btn-profile"
href="/profile">Profile</a></li>

            {{else}}

            <li><a class="btn-login"
href="/login">Login</a></li>

            <li><a class="btn-register"
href="/register">Register</a></li>

            {{end}}

        </ul>

    </nav>

</header>

<main>

    <section class="hero-recommendations">
```

<h1>Hero Recommendations</h1>

<h2>Recommended Picks</h2>

Pick 1: Phantom Assassin - Strong against
current enemy heroes.

Pick 2: Lion - Provides needed disable and
burst.

<!-- Добавь больше рекомендаций -->

<h2>Counter Picks</h2>

Counter Pick: Nyx Assassin - Excellent
against enemy spellcasters.

</section>

</main>

<footer>

<p>© 2024 Dota2 Match Prediction</p>

</footer>


```
</body>

</html>
```

web\templates\register.html:

```
<!DOCTYPE html>

<html lang="en">

    <head>

        <meta charset="UTF-8" />

        <meta name="viewport" content="width=device-width, initial-
scale=1.0" />

        <title>Register</title>

        <link rel="stylesheet" href="./web/static/style.css" />

    </head>

    <body>

        <header class="page-header">

            <nav>

                <ul>

                    <li><a href="/">Home</a></li>

                    <li><a href="/prediction">Match
Prediction</a></li>

                    <li><a href="/statistics">Statistics</a></li>

                    <li><a href="/team-analysis">Team
Analysis</a></li>
```

```
Recommendations</a></li>
    <li><a href="/recommendations">Hero
```

```
    {{if .IsLoggedIn}}
```

```
    <li><a class="btn-profile"
href="/profile">Profile</a></li>
```

```
    {{else}}
```

```
    <li><a class="btn-login"
href="/login">Login</a></li>
```

```
    <li><a class="btn-register"
href="/register">Register</a></li>
```

```
    {{end}}
```

```
</ul>
```

```
</nav>
```

```
</header>
```

```
<main>
```

```
    <section class="form-container">
```

```
        <form action="/register" method="POST">
```

```
            <div class="form-group">
```

```
                <label for="email">Email</label>
```

```
                <input type="email" id="email"
name="email" required />
```

```
            </div>
```

```
            <div class="form-group">
```

```
                <label for="password">Password</label>
```

```
name="password" required />
</div>
```

```
<div class="form-group">
  <label for="confirm-password">Confirm
  Password</label>
  <input
    type="password"
    id="confirm-password"
    name="confirm-password"
    required
  />
</div>
```

```
<button type="submit" class="btn-
submit">Register</button>
</form>
```

```
<p class="alt-option">
  Already have an account? <a href="/login">Log
  in</a>
```

```
</p>
{{if .ErrorMessage}}
<div class="error-message">{{.ErrorMessage}}</div>
{{end}}
```

```
        </section>

    </main>

</body>

</html>
```

web\templates\statistics.html:

```
<!DOCTYPE html>

<html lang="en">

    <head>

        <meta charset="UTF-8" />

        <meta name="viewport" content="width=device-width, initial-
scale=1.0" />

        <title>Dota2 Match Prediction - Statistics</title>

        <link rel="stylesheet" href="./web/static/style.css" />

    </head>

    <body>

        <header>

            <nav>

                <ul>

                    <li><a href="/">Home</a></li>

                    <li><a href="/prediction">Match
Prediction</a></li>

                    <li><a href="/statistics">Statistics</a></li>
```



```

                                <th>Winrate</th>

                                </tr>

                                </thead>

                                <tbody>

                                    {{range .HeroesWinrates}}

                                        <tr>

                                            <td>{{.Hero.Value}}</td>

                                            <td>{{.Winrate}}%</td>

                                        </tr>

                                    {{end}}

                                </tbody>

                                </table>

                                </section>

                                </main>

                                <footer>

                                    <p>&copy; 2024 Dota2 Match Prediction</p>

                                </footer>

                                </body>

                                </html>
```

web\templates\teams.html:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
    <head>
```

```
        <meta charset="UTF-8" />
```

```
        <meta name="viewport" content="width=device-width, initial-  
scale=1.0" />
```

```
        <title>Team Analysis</title>
```

```
        <link rel="stylesheet" href="/web/static/style.css" />
```

```
    </head>
```

```
    <body>
```

```
        <header>
```

```
            <nav>
```

```
                <ul>
```

```
                    <li><a href="/">Home</a></li>
```

```
                    <li><a href="/prediction">Match  
Prediction</a></li>
```

```
                    <li><a href="/statistics">Statistics</a></li>
```

```
                    <li><a href="/team-analysis">Team  
Analysis</a></li>
```

```
                    <li><a href="/recommendations">Hero  
Recommendations</a></li>
```

```
                {{if .IsLoggedIn}}
```

```
                    <li><a class="btn-profile"  
href="/profile">Profile</a></li>
```

```
                {{else}}
```

```
        <li><a class="btn-login"
href="/login">Login</a></li>
```

```
        <li><a class="btn-register"
href="/register">Register</a></li>
```

```
    {{end}}
```

```
</ul>
```

```
</nav>
```

```
</header>
```

```
<main>
```

```
    <section class="team-analysis">
```

```
        <table>
```

```
            <thead>
```

```
                <tr>
```

```
                    <th>Team</th>
```

```
                    <th>Player 1</th>
```

```
                    <th>Player 2</th>
```

```
                    <th>Player 3</th>
```

```
                    <th>Player 4</th>
```

```
                    <th>Player 5</th>
```

```
                </tr>
```

```
            </thead>
```

```
            <tbody>
```

```
                {{range .TeamsRoaters}}
```



```
<tr>

    <td>{{.Team.Value}}</td>

    {{range .Players}}

    <td>{{.Value}}</td>

    {{end}}

</tr>

{{end}}

</tbody>

</table>

</section>

</main>

<footer>

    <p>&copy; 2024 Dota2 Match Prediction</p>

</footer>

</body>

</html>
```